

DATA 260 Homework 5

Priyank Mehta

SJSU ID: 019152562

Municipal Transit Incidents

Repository: <https://github.com/priyankmehta5/data260-2562>

Pushed implementation commit: 0075108ef6118b79bfda3560c866e1ade6e10ee0

1. Personal Configuration

This report documents the HW5 extension of the municipal transit incident application. The work adds a related transit-agency entity, validated CRUD APIs, Redux Toolkit client state, two MCP servers, retry experiments, an offline tool entry point, and a bounded Ollama-based agent loop.

Value	Configuration
SID4	2562
PORT_BASE	8762
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Model	qwen3:1.7b
Embedding model	sentence-transformers/all-MiniLM-L6-v2

2. Part 1 Database API and Redux Client

The backend adds TransitAgency, links incidents through a restricted foreign key, validates request data with Pydantic, and exposes CRUD and relationship routes. The React client uses Redux Toolkit slices and Axios thunks while retaining session cookies.

```

models.py X
backend > models.py > ...
16 class TransitAgency(Base):
17     __tablename__ = "transit_agencies"
18
19     id: Mapped[int] = mapped_column(
20         Integer,
21         primary_key=True,
22         autoincrement=True,
23     )
24     name: Mapped[str] = mapped_column(
25         String(150),
26         nullable=False,
27     )
28     jurisdiction: Mapped[str] = mapped_column(
29         String(150),
30         nullable=False,
31     )
32     agency_code: Mapped[str] = mapped_column(
33         String(30),
34         unique=True,
35         nullable=False,
36         index=True,
37     )
38     created_at: Mapped[datetime] = mapped_column(
39         DateTime,
40         nullable=False,
41         server_default=func.now(),
42     )
43     updated_at: Mapped[datetime] = mapped_column(
44         DateTime,
45         nullable=False,
46         server_default=func.now(),
47         onupdate=func.now(),
48     )
49
50     incidents: Mapped[list["Incident"]] = relationship(
51         back_populates="agency",
52         passive_deletes=True,
53     )
54
55
56 class Incident(Base):
57     __tablename__ = "incidents"
58
59     id: Mapped[int] = mapped_column(
60         Integer,
61         primary_key=True,
62         autoincrement=True,
63     )
64     title: Mapped[str] = mapped_column(
65         String(100),
66         nullable=False,
67     )
68     description: Mapped[str] = mapped_column(
69         Text,
70         nullable=False,
71     )
72     incident_code: Mapped[str] = mapped_column(
73         String(30),
74         unique=True,
75         nullable=False,
76         index=True,
77     )
78     severity: Mapped[int] = mapped_column(
79         Integer,
80         nullable=False,
81         default=1,
82         server_default="1",
83     )
84     agency_id: Mapped[int] = mapped_column(
85         ForeignKey(
86             "transit_agencies.id",
87             ondelete="RESTRICT",
88         ),
89         nullable=False,
90         index=True,
91     )

```

Part 1 - Database models: related TransitAgency entity and Incident foreign-key relationship

```

models.py U schemas.py U X
backend > schemas.py > IncidentResponse

27 class AgencyCreate(BaseModel):
28     name: str = Field(min_length=2, max_length=150)
29     jurisdiction: str = Field(min_length=2, max_length=150)
30     agency_code: str = Field(
31         min_length=3,
32         max_length=30,
33         pattern=CODE_PATTERN,
34     )
35
36     @field_validator("name", "jurisdiction")
37     @classmethod
38     def strip_text(cls, value: str) -> str:
39         return value.strip()
40
41     @field_validator("agency_code", mode="before")
42     @classmethod
43     def normalize_code(cls, value: str) -> str:
44         return str(value).strip().upper()
45
46
47 class AgencyUpdate(BaseModel):
48     name: str | None = Field(
49         default=None,
50         min_length=2,
51         max_length=150,
52     )
53     jurisdiction: str | None = Field(
54         default=None,
55         min_length=2,
56         max_length=150,
57     )
58     agency_code: str | None = Field(
59         default=None,
60         min_length=3,
61         max_length=30,
62         pattern=CODE_PATTERN,
63     )
64
65     @field_validator("name", "jurisdiction")
66     @classmethod
67     def strip_optional_text(
68         cls,
69         value: str | None,
70     ) -> str | None:
71         return value.strip() if value is not None else None
72
73     @field_validator("agency_code", mode="before")
74     @classmethod
75     def normalize_optional_code(
76         cls,
77         value: str | None,
78     ) -> str | None:
79         if value is None:
80             return None
81         return str(value).strip().upper()

```

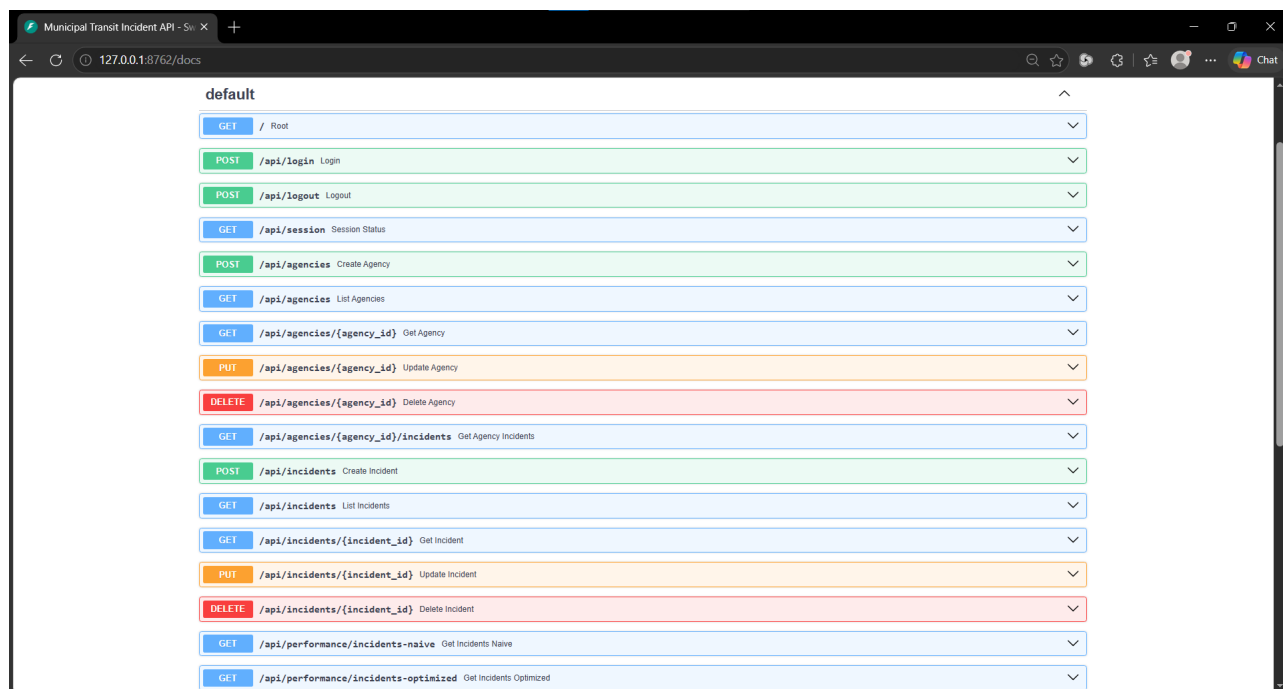
Part 1 - Pydantic validation for the related transit-agency entity, including normalized unique agency codes

```
routes_hw05.py U X  models.py U  schemas.py U
backend > routes_hw05.py > ...
11  # ----- Transit agencies -----
12
13  @router.post(
14      "/agencies",
15      response_model=schemas.AgencyResponse,
16      status_code=201,
17  )
18  def create_agency(
19      data: schemas.AgencyCreate,
20      database: Session = Depends(get_db),
21  ):
22      return crud.create_agency(database, data)
23
24
25  @router.get(
26      "/agencies",
27      response_model=list[schemas.AgencyResponse],
28  )
29  def list_agencies(
30      skip: int = Query(default=0, ge=0),
31      limit: int = Query(default=50, ge=1, le=200),
32      database: Session = Depends(get_db),
33  ):
34      return crud.list_agencies(
35          database,
36          skip,
37          limit,
38      )
39
40
41  @router.get(
42      "/agencies/{agency_id}",
43      response_model=schemas.AgencyResponse,
44  )
45  def get_agency(
46      agency_id: int,
47      database: Session = Depends(get_db),
```

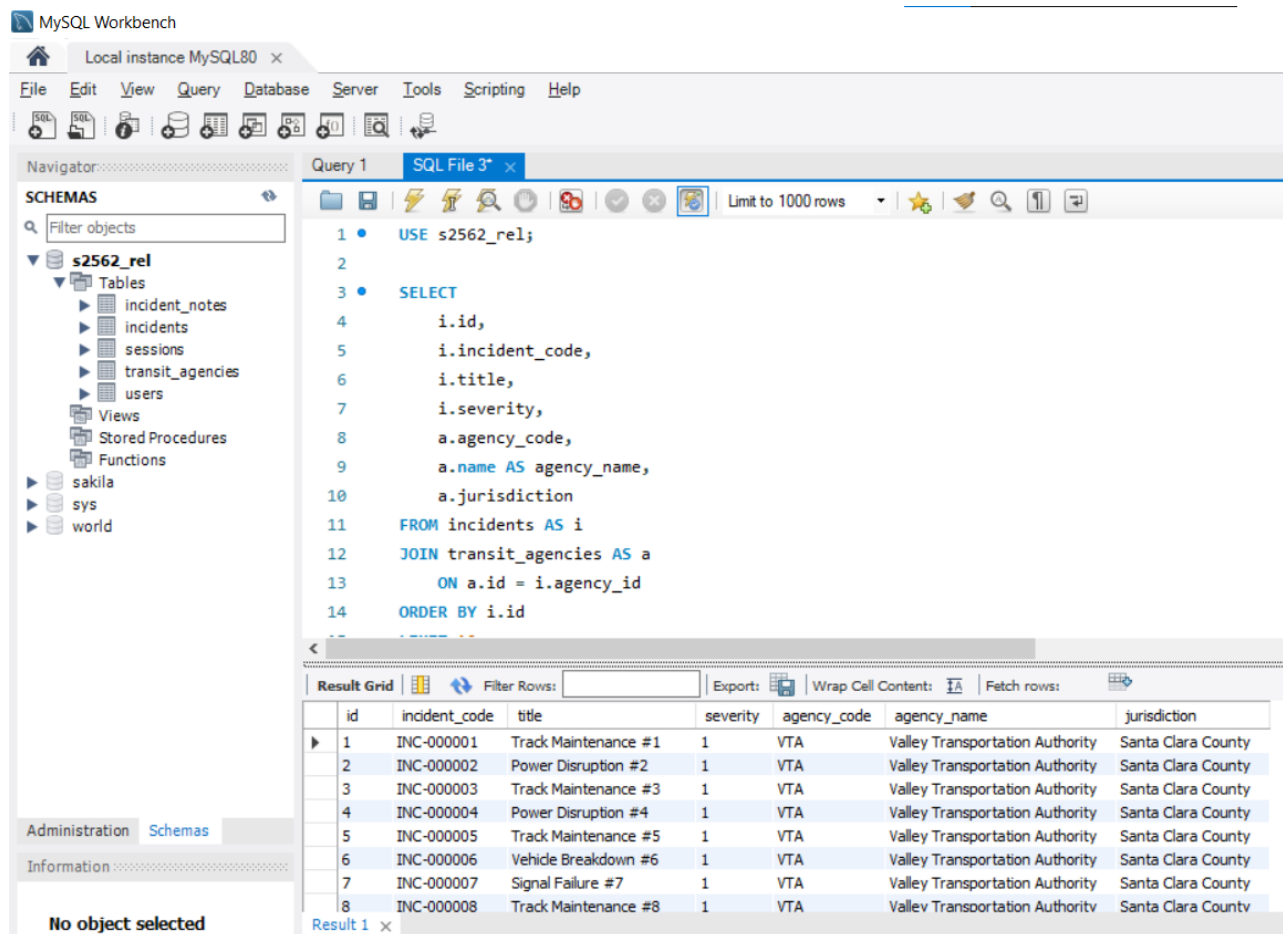
Part 1 - Paginated CRUD API routes for the related TransitAgency entity

```
routes_hw05.py U X  models.py U  schemas.py U
backend > routes_hw05.py > ...
84 def get_agency_incidents(
85     agency_id: int,
86     database: Session = Depends(get_db),
87 ):
88     return crud.incidents_by_agency(
89         database,
90         agency_id,
91     )
92
93
94 # ----- Transit incidents -----
95
96 @router.post(
97     "/incidents",
98     response_model=schemas.IncidentResponse,
99     status_code=201,
100 )
101 def create_incident(
102     data: schemas.IncidentCreate,
103     database: Session = Depends(get_db),
104 ):
105     return crud.create_incident(database, data)
106
107
108 @router.get(
109     "/incidents",
110     response_model=list[schemas.IncidentResponse],
111 )
112 def list_incidents(
113     skip: int = Query(default=0, ge=0),
114     limit: int = Query(default=50, ge=1, le=200),
115     database: Session = Depends(get_db),
116 ):
117     return crud.list_incidents(
118         database,
119         skip,
120         limit,
```

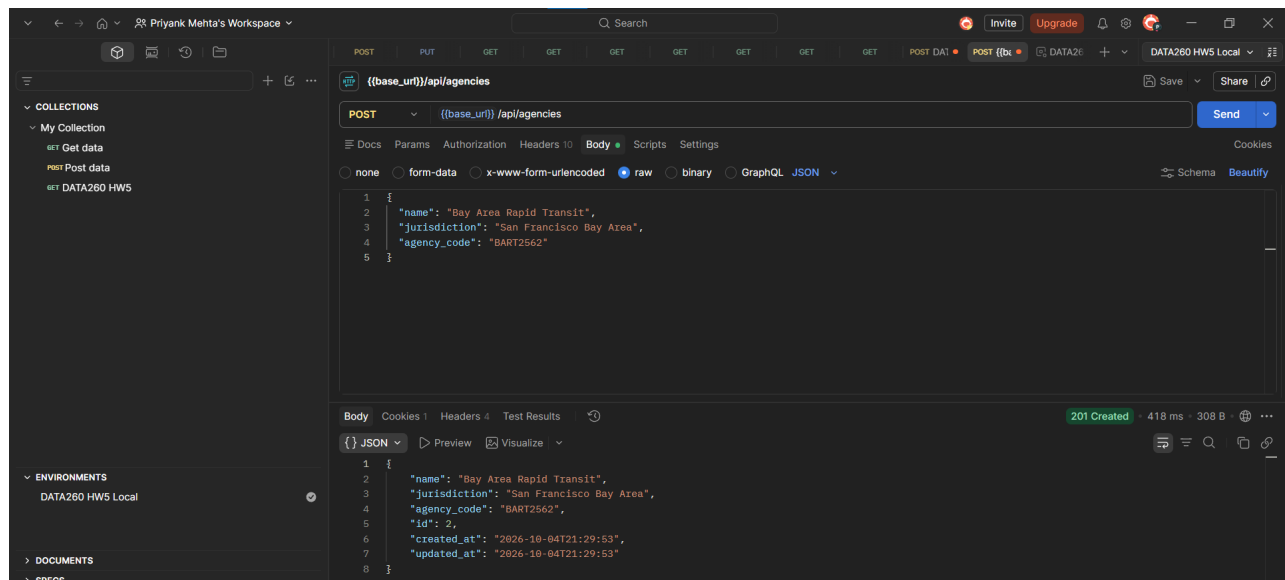
Part 1 - Agency-to-incidents relationship endpoint and incident CRUD routes



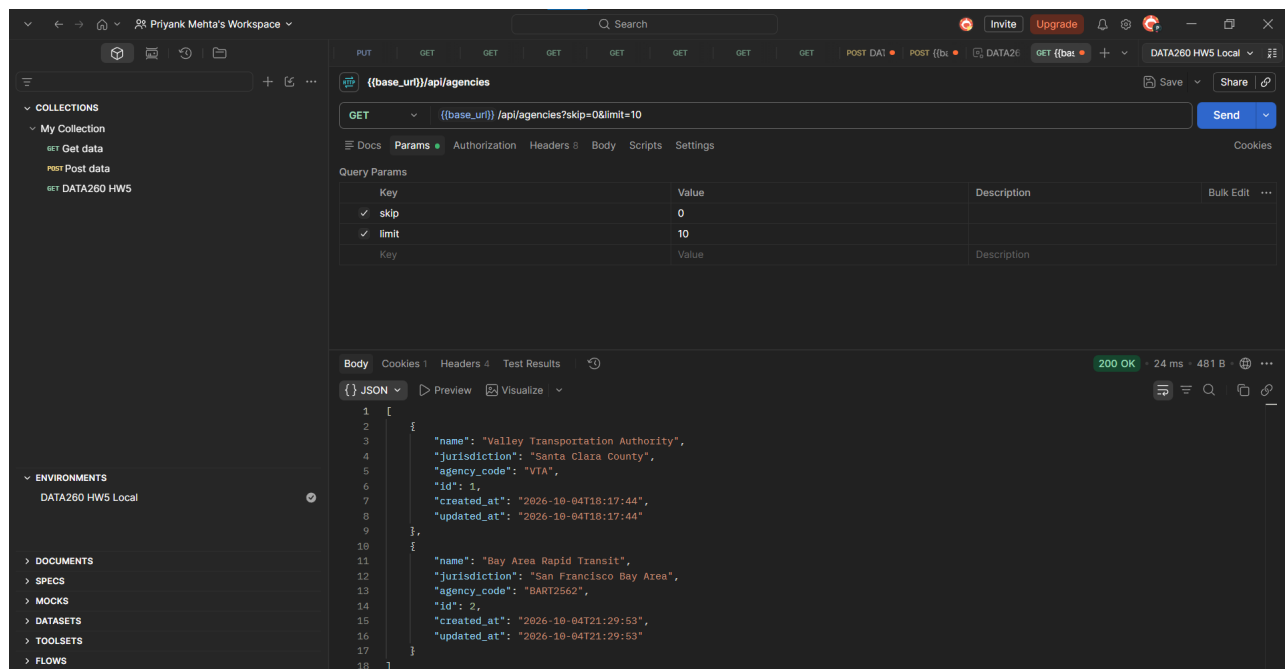
Part 1 - FastAPI CRUD and relationship endpoints for transit agencies and incidents



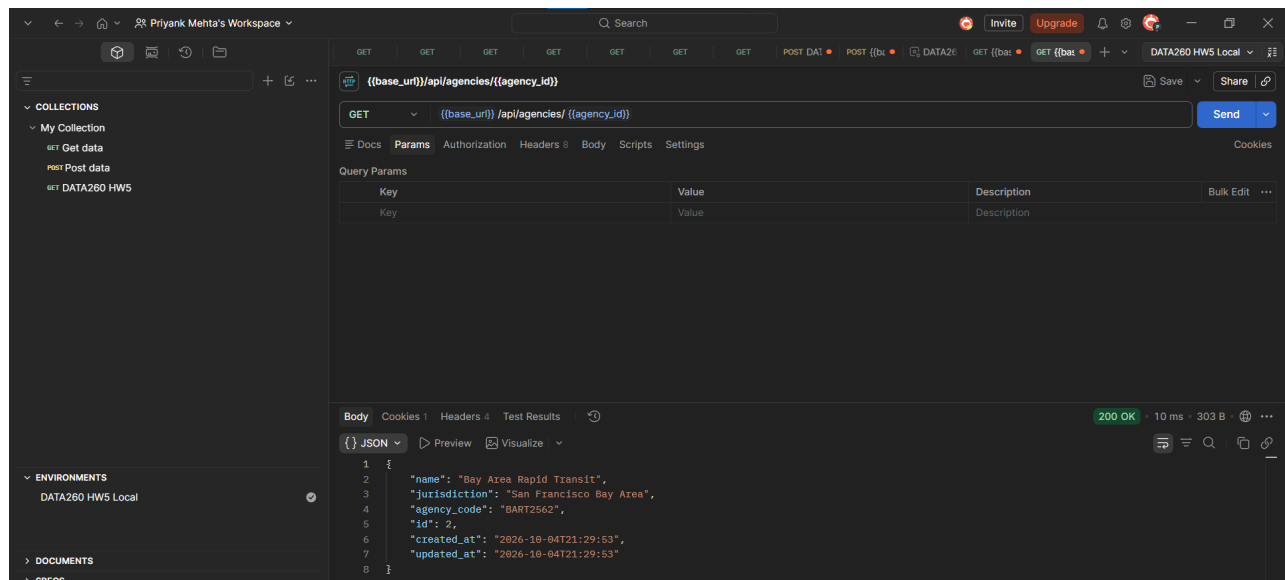
Part 1 - MySQL database showing transit agencies and their related municipal transit incidents



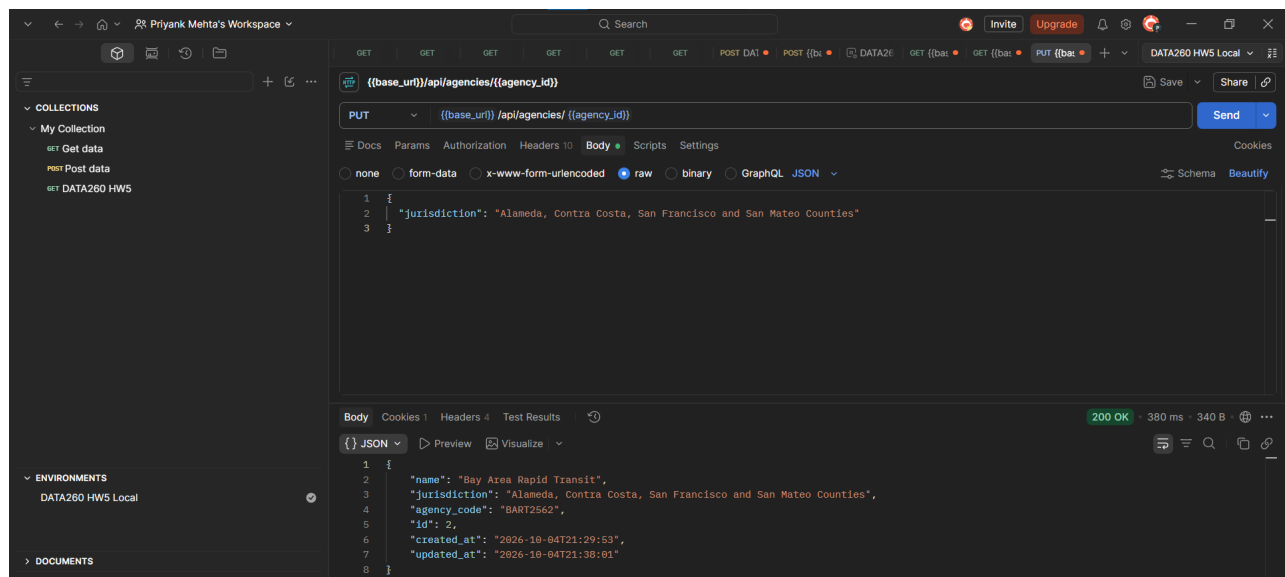
Part 1 - Creating a related transit-agency record



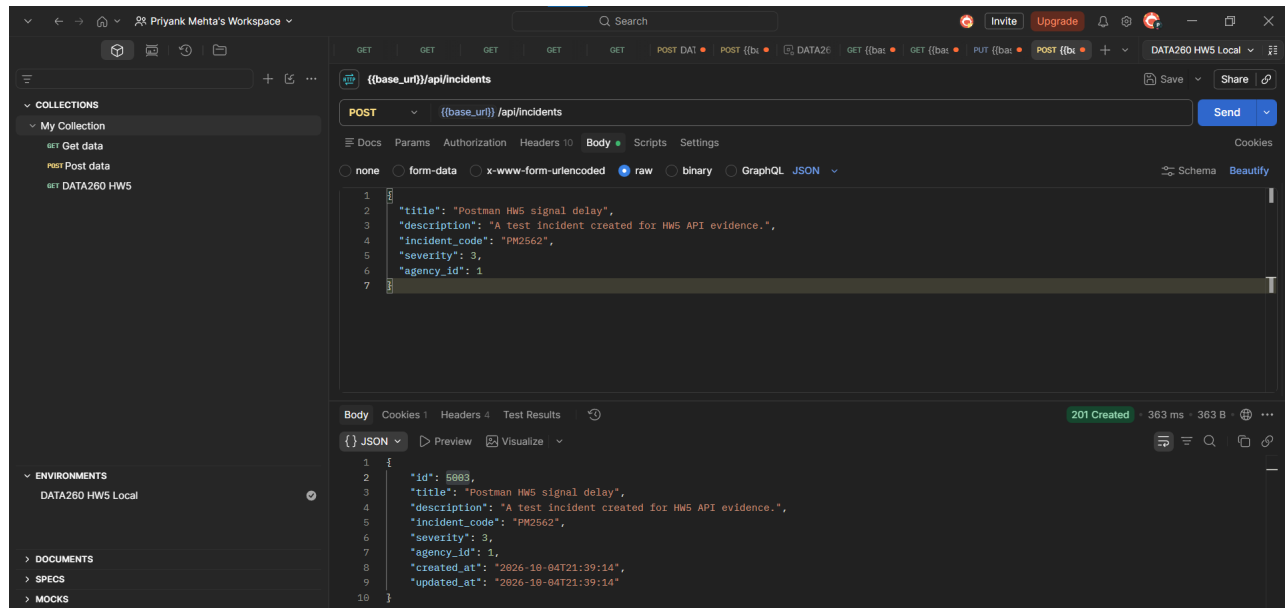
Part 1 - Retrieving transit agencies with pagination



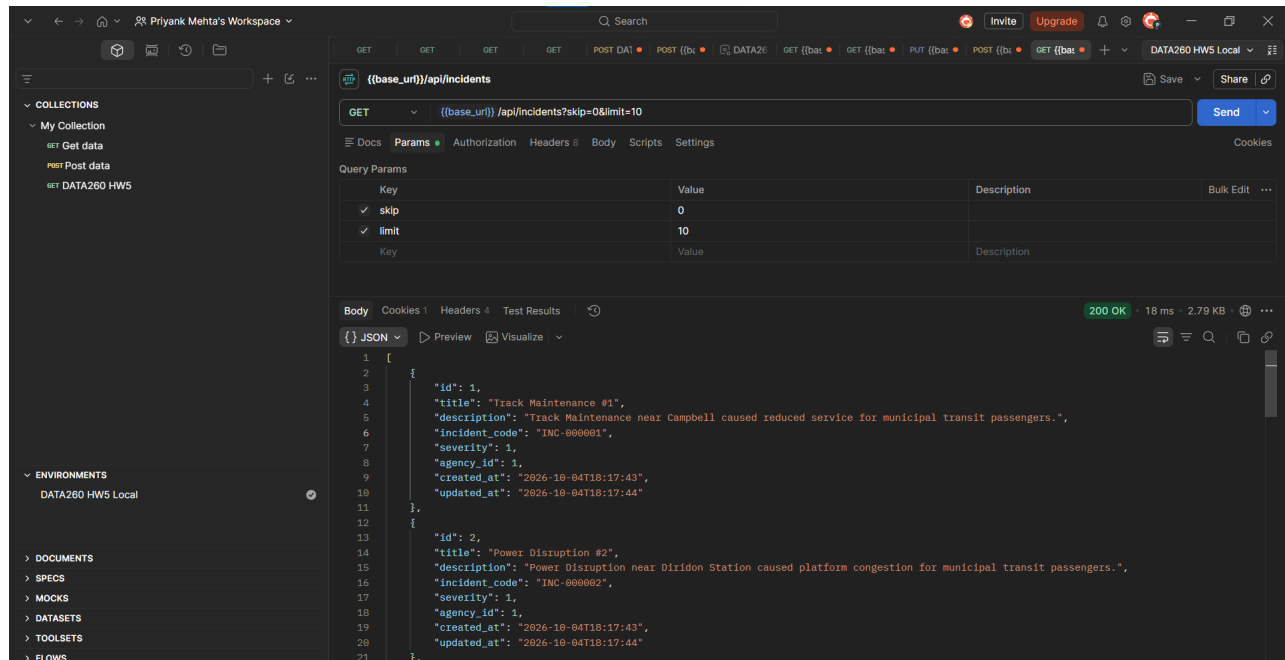
Part 1 - Retrieving an individual transit agency



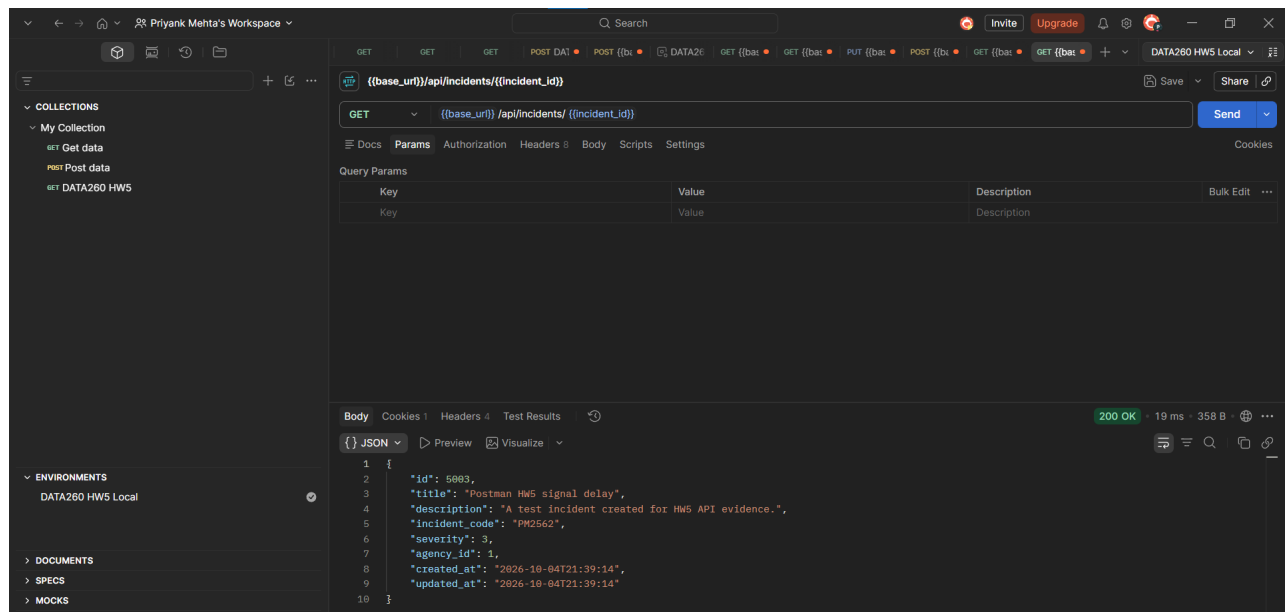
Part 1 - Updating a transit-agency record



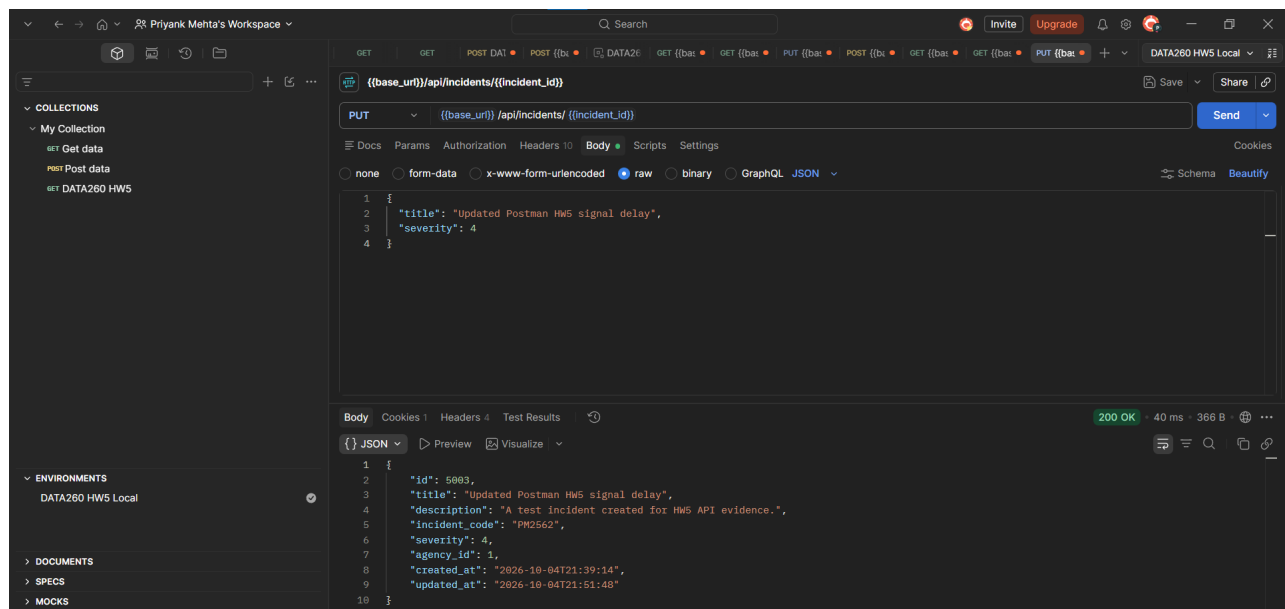
Part 1 - Creating a primary transit-incident record



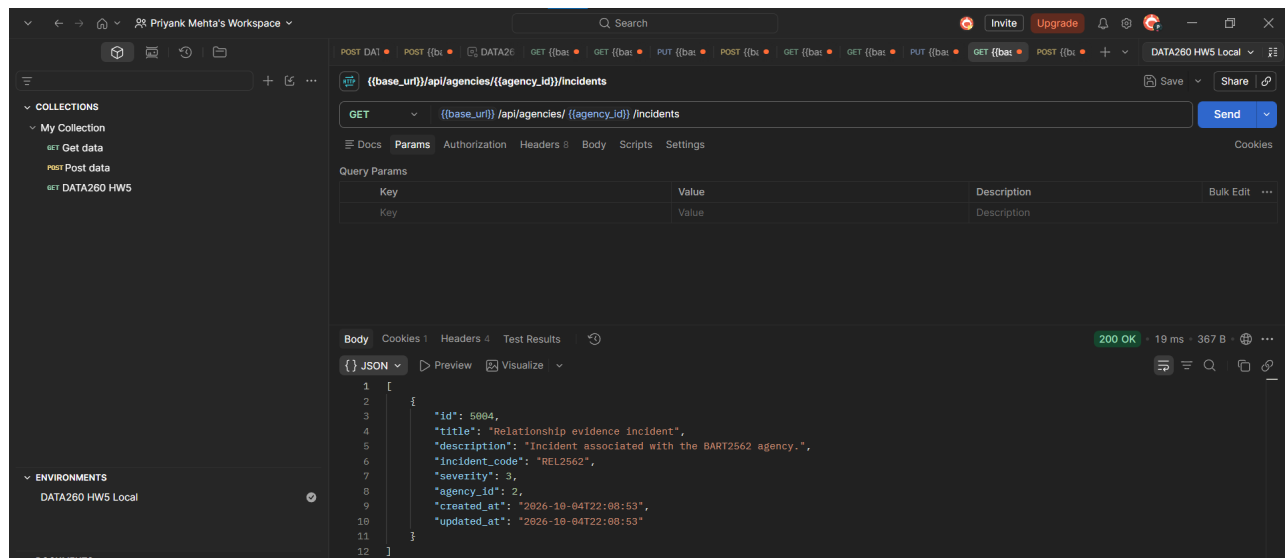
Part 1 - Retrieving transit incidents with pagination



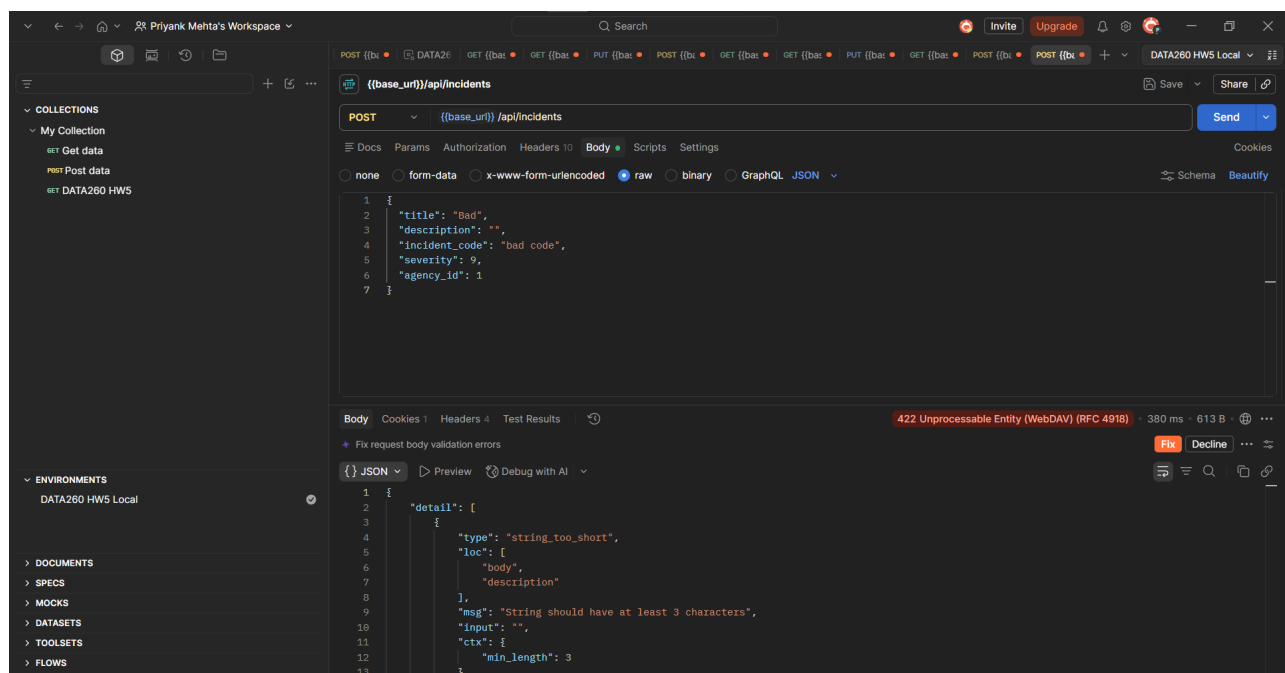
Part 1 - Retrieving an individual transit incident



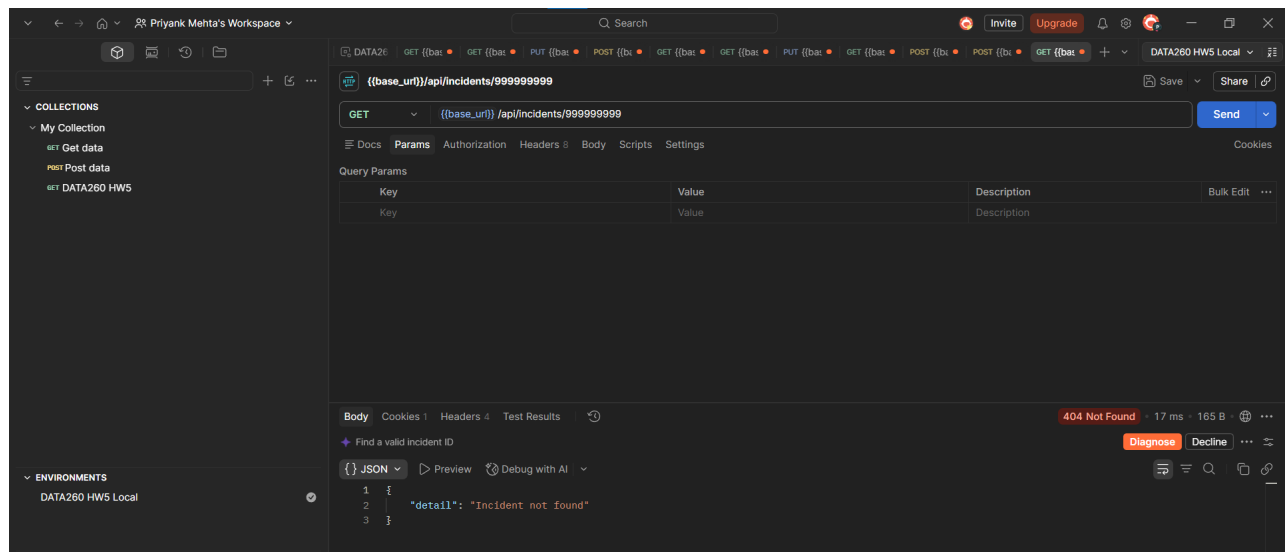
Part 1 - Updating a transit-incident record



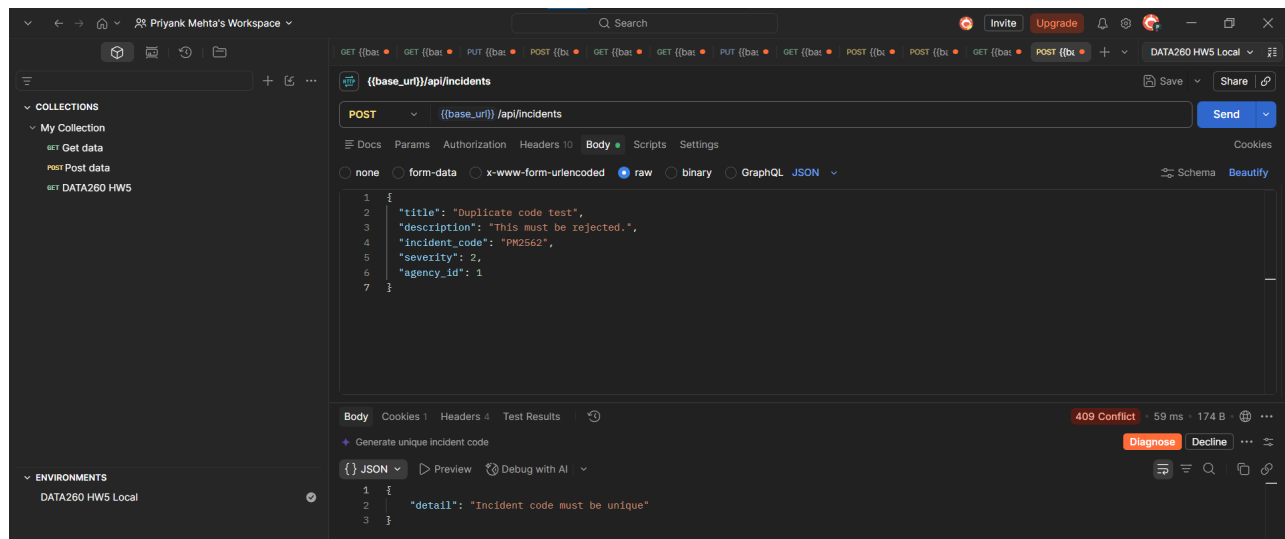
Part 1 - Retrieving incidents for a transit agency



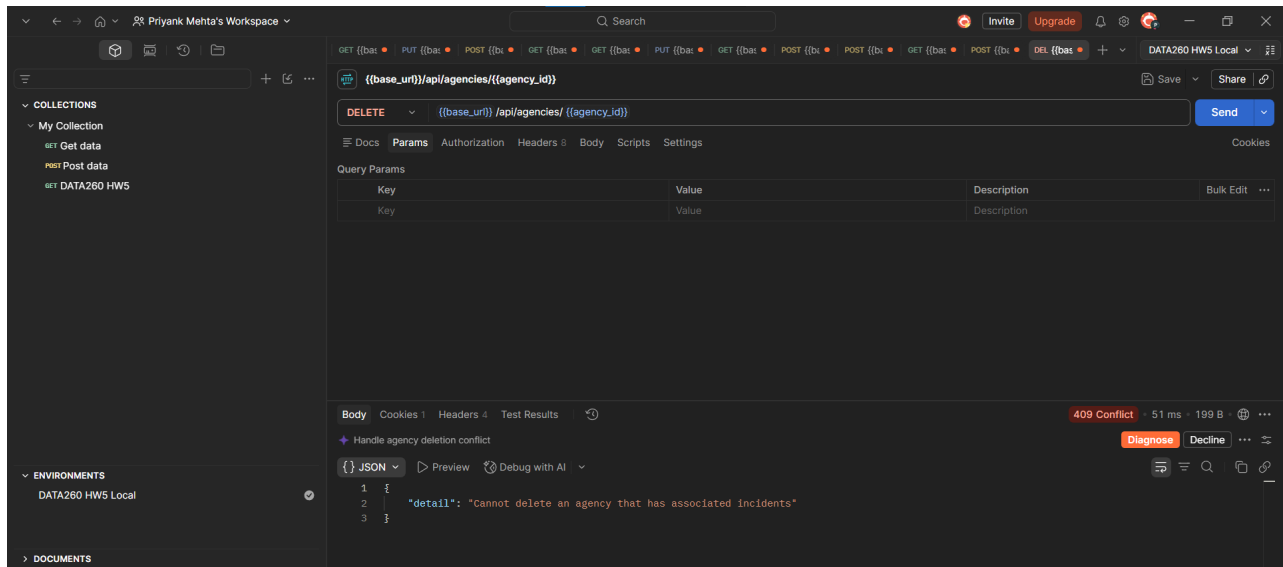
Part 1 - Rejecting invalid incident input



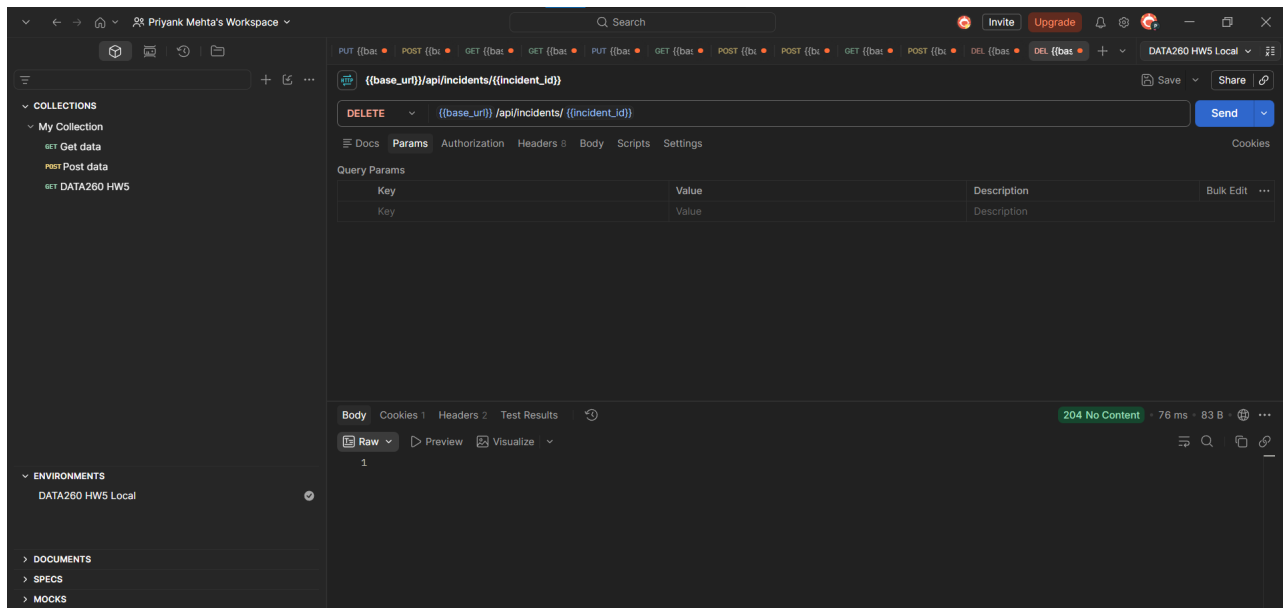
Part 1 - Handling a missing incident



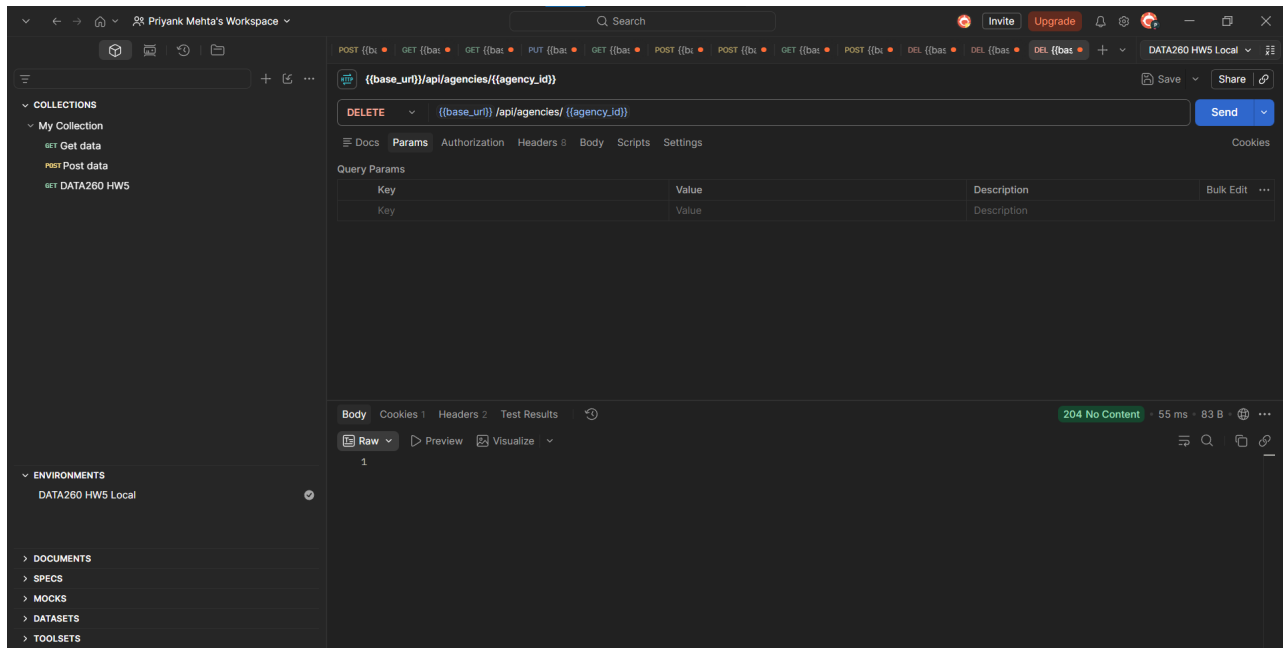
Part 1 - Rejecting a duplicate incident code



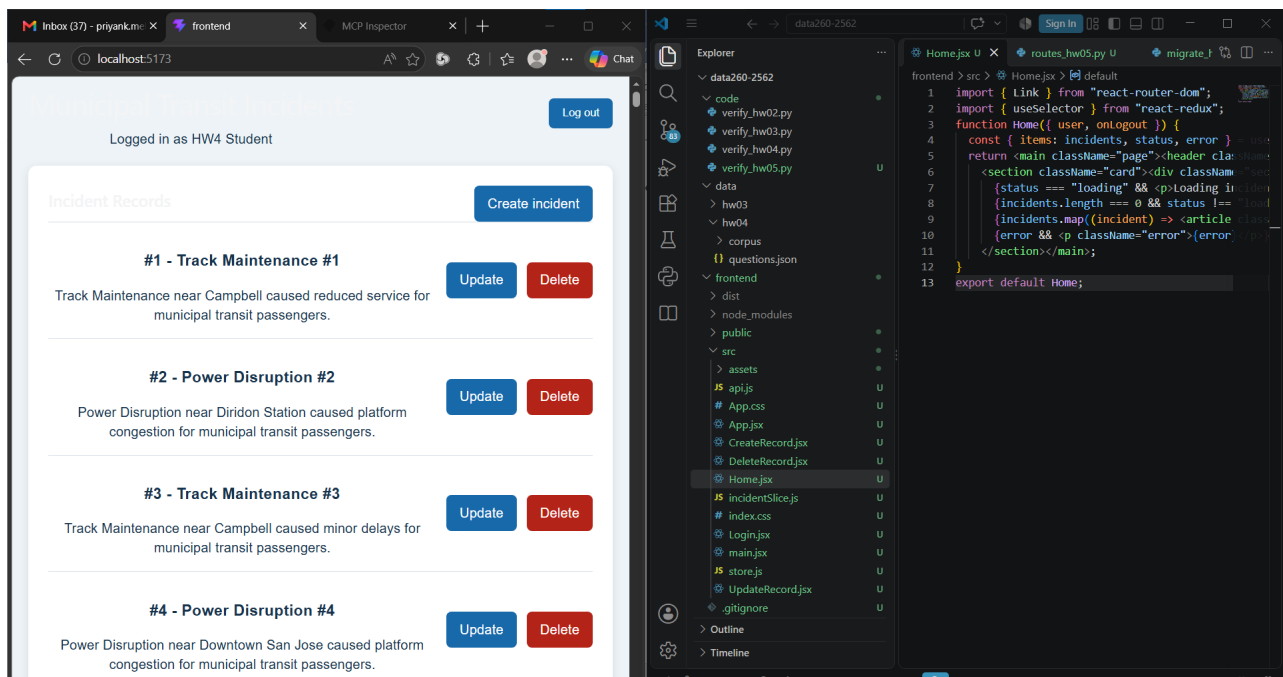
Part 1 - Protecting an agency referenced by incidents



Part 1 - Deleting a transit incident



Part 1 - Deleting an unused transit agency



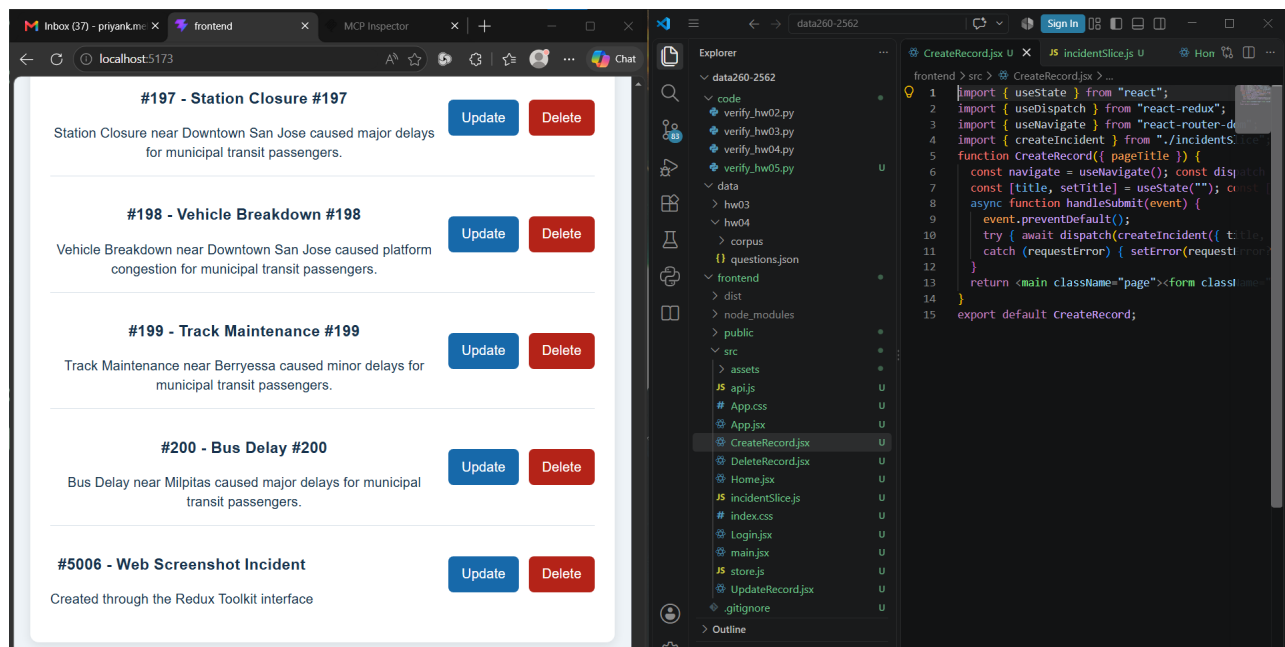
Part 1 - Redux Home page displaying incident records

```

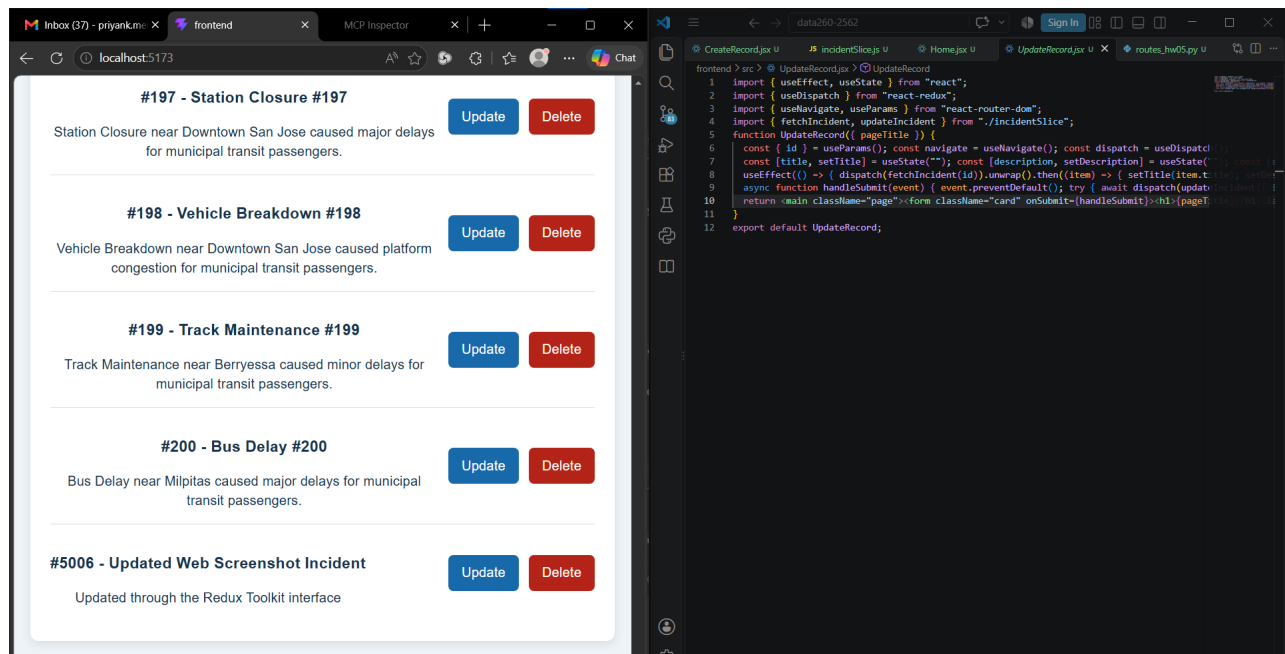
1 import { createAsyncThunk, createSlice } from "@reduxjs/toolkit";
2 import api from "../api";
3
4 const message = (error) => error.response?.data?.detail || error.message || "Request failed";
5 export const fetchIncidents = createAsyncThunk("incidents/fetch", async (_, { rejectWithValue }) => {
6   try { return (await api.get("/api/incidents?skip=0&limit=200")).data; }
7   catch (error) { return rejectWithValue(message(error)); }
8 });
9 export const fetchIncident = createAsyncThunk("incidents/fetchOne", async (id, { rejectWithValue }) => {
10   try { return (await api.get("/api/incidents/" + id)).data; }
11   catch (error) { return rejectWithValue(message(error)); }
12 });
13 export const createIncident = createAsyncThunk("incidents/create", async (payload, { rejectWithValue }) => {
14   try { return (await api.post("/api/incidents", payload)).data; }
15   catch (error) { return rejectWithValue(message(error)); }
16 });
17 export const updateIncident = createAsyncThunk("incidents/update", async ({ id, changes }, { rejectWithValue }) => {
18   try { return (await api.put("/api/incidents/" + id, changes)).data; }
19   catch (error) { return rejectWithValue(message(error)); }
20 });
21 export const deleteIncident = createAsyncThunk("incidents/delete", async (id, { rejectWithValue }) => {
22   try { await api.delete("/api/incidents/" + id); return id; }
23   catch (error) { return rejectWithValue(message(error)); }
24 });
25
26 const slice = createSlice({
27   name: "incidents",
28   initialState: { items: [], status: "idle", error: null },
29   reducers: {},
30   extraReducers: (builder) => {
31     builder.addCase(fetchIncidents.pending, (state) => { state.status = "loading"; state.error = null; });
32     builder.addCase(fetchIncidents.fulfilled, (state, action) => { state.status = "succeeded"; state.items = action.payload; });
33     builder.addCase(fetchIncidents.rejected, (state, action) => { state.status = "failed"; state.error = action.payload; });
34     builder.addCase(createIncident.fulfilled, (state, action) => { state.items.push(action.payload); });
35     builder.addCase(updateIncident.fulfilled, (state, action) => {
36       const index = state.items.findIndex((item) => item.id === action.payload.id);
37       if (index >= 0) state.items[index] = action.payload;
38     });
39     builder.addCase(deleteIncident.fulfilled, (state, action) => {
40       state.items = state.items.filter((item) => item.id !== action.payload);
41     });
42   },
43 });
44 export default slice.reducer;

```

Part 1 - Redux Create feature successfully adding an incident



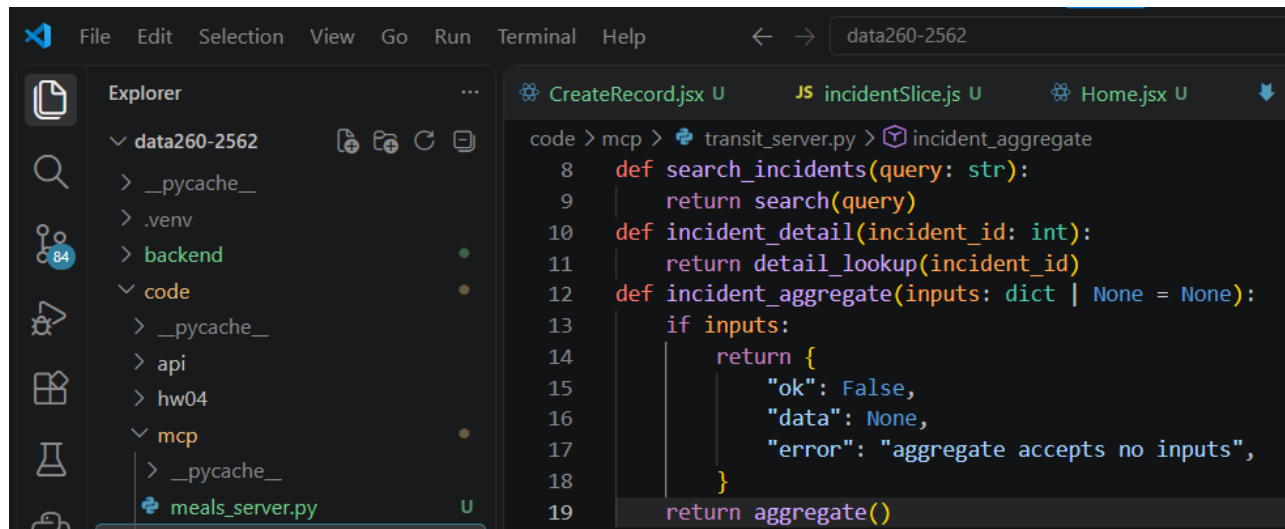
Part 1 - Redux Update feature modifying an incident



Part 1 - Redux Delete feature removing an incident

3. Part 2 MCP Tool Servers

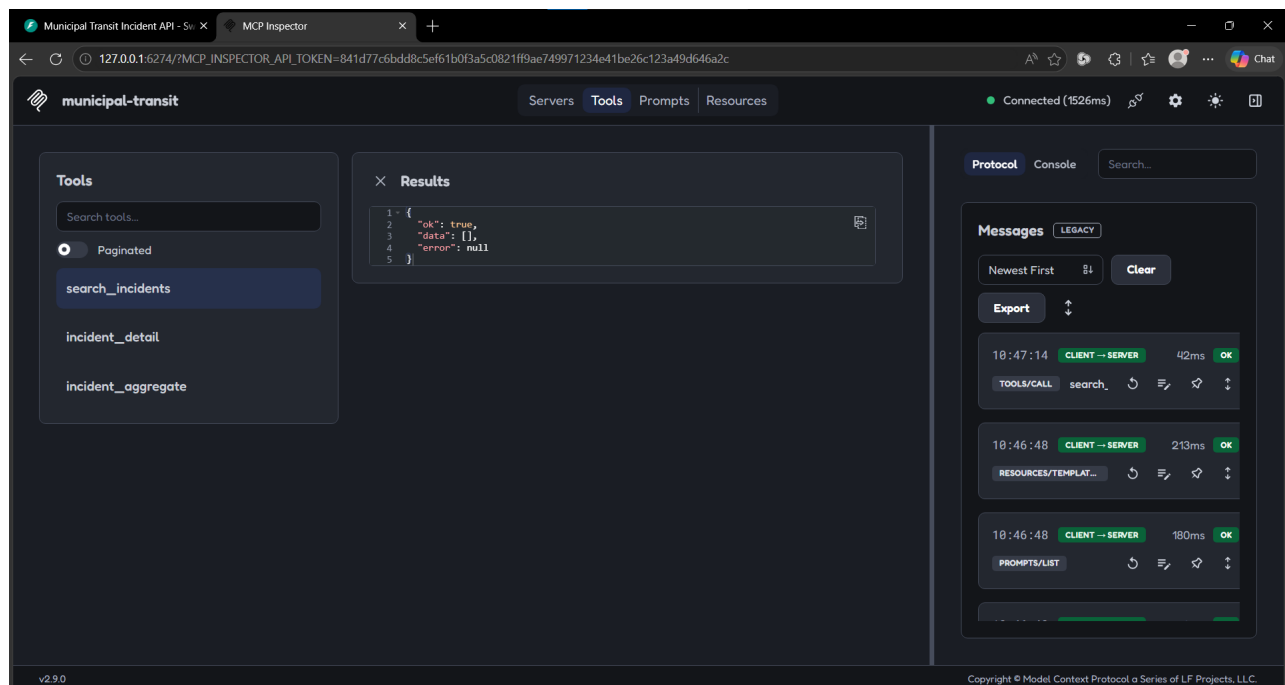
TheMealDB provides four recipe tools. The municipal-transit server provides search, detail lookup, and aggregate operations using the common {ok, data, error} envelope. Inspector evidence shows successful and rejected calls.



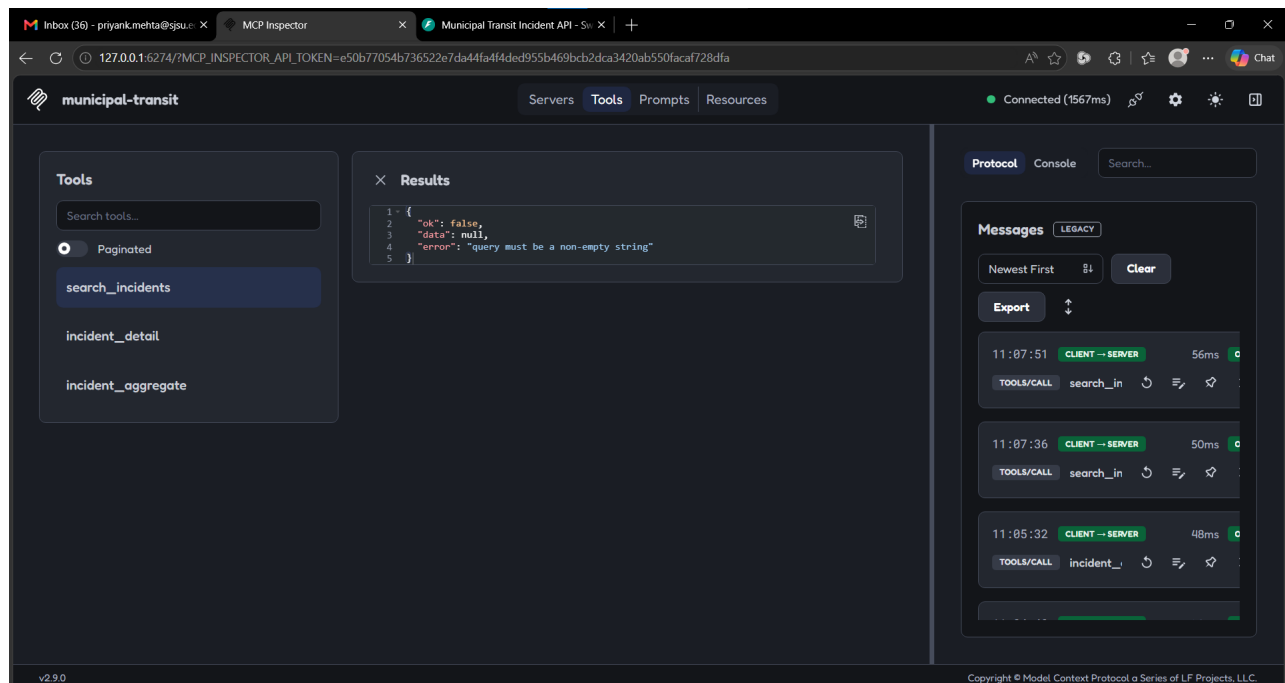
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like `data260-2562`, `code`, `api`, `hw04`, and `mcp`. The code editor shows the implementation of the `incident_aggregate` tool in `transit_server.py`. The code defines three functions: `search_incidents`, `incident_detail`, and `incident_aggregate`. The `incident_aggregate` function is currently implemented to return a response indicating that it accepts no inputs.

```
code > mcp > transit_server.py > incident_aggregate
8 def search_incidents(query: str):
9     return search(query)
10 def incident_detail(incident_id: int):
11     return detail_lookup(incident_id)
12 def incident_aggregate(inputs: dict | None = None):
13     if inputs:
14         return {
15             "ok": False,
16             "data": None,
17             "error": "aggregate accepts no inputs",
18         }
19     return aggregate()
```

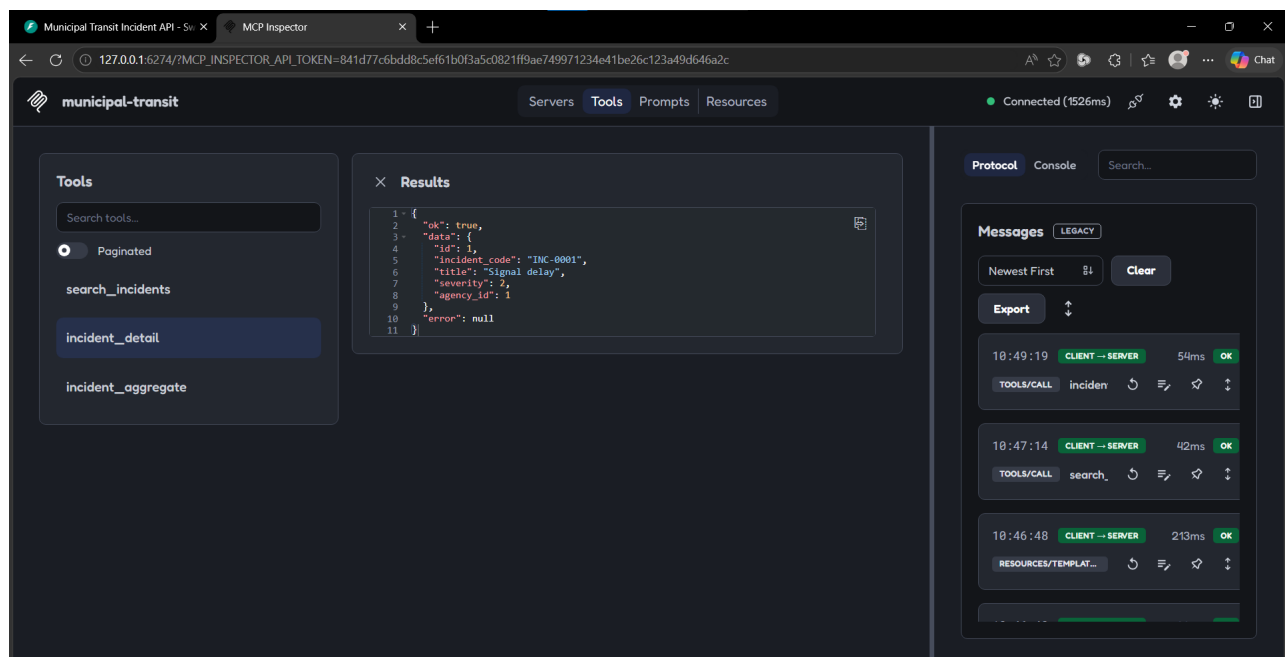
Part 2B - Municipal transit MCP tools and response-envelope validation



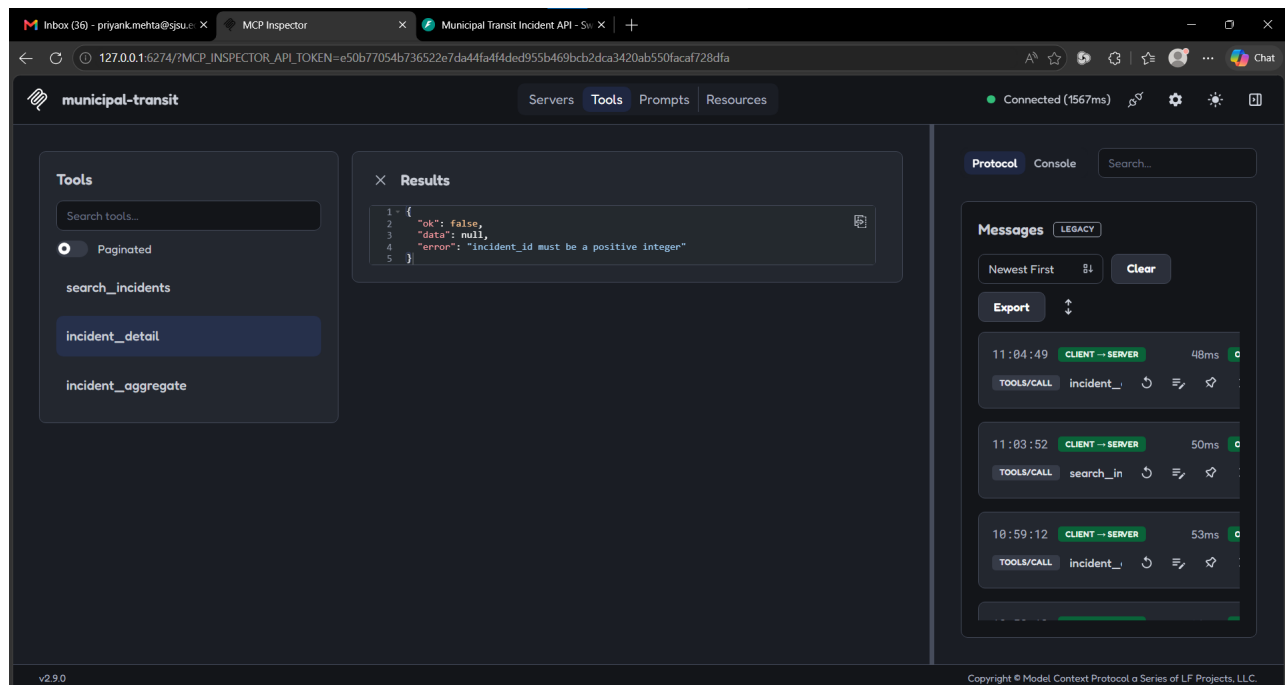
Part 2B - Successful municipal transit search



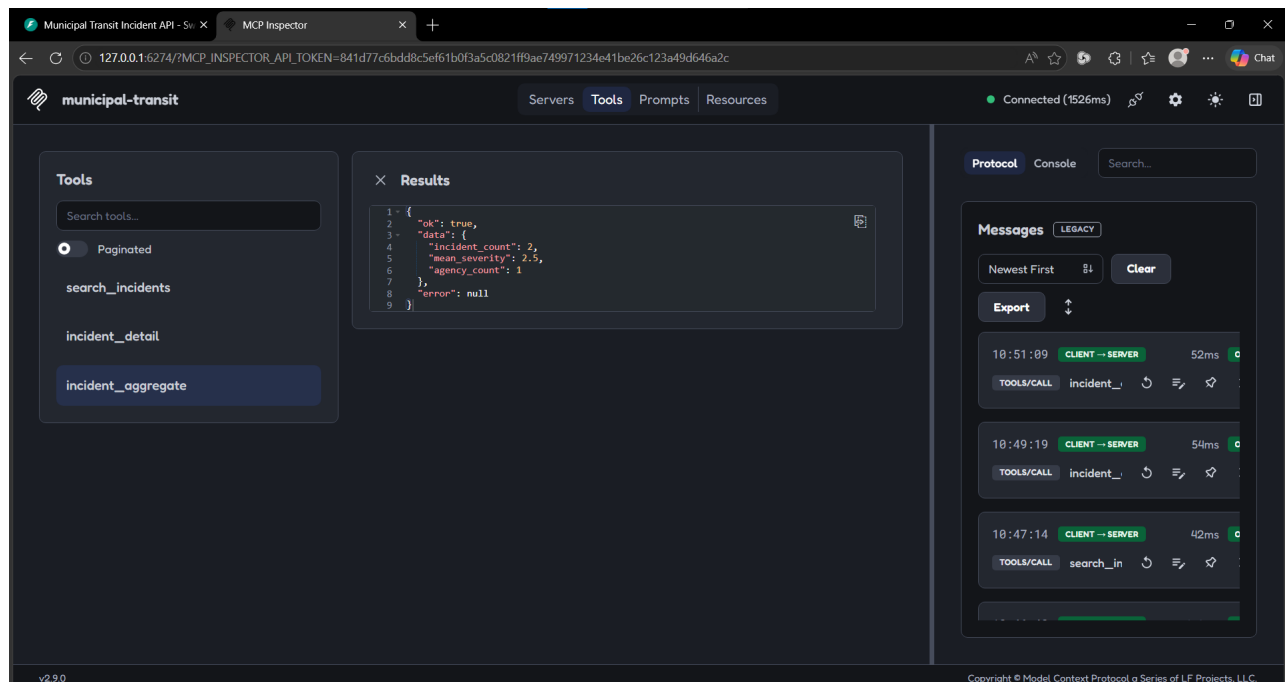
Part 2B - Rejected municipal transit search



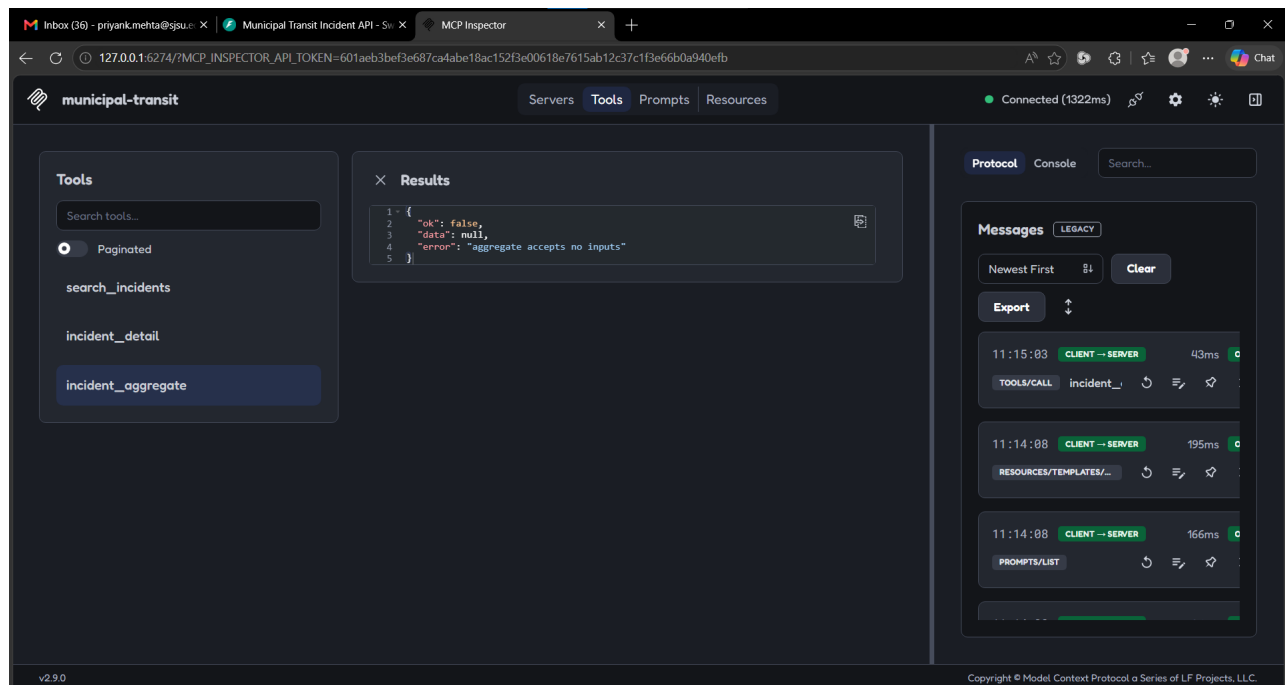
Part 2B - Successful municipal transit detail lookup



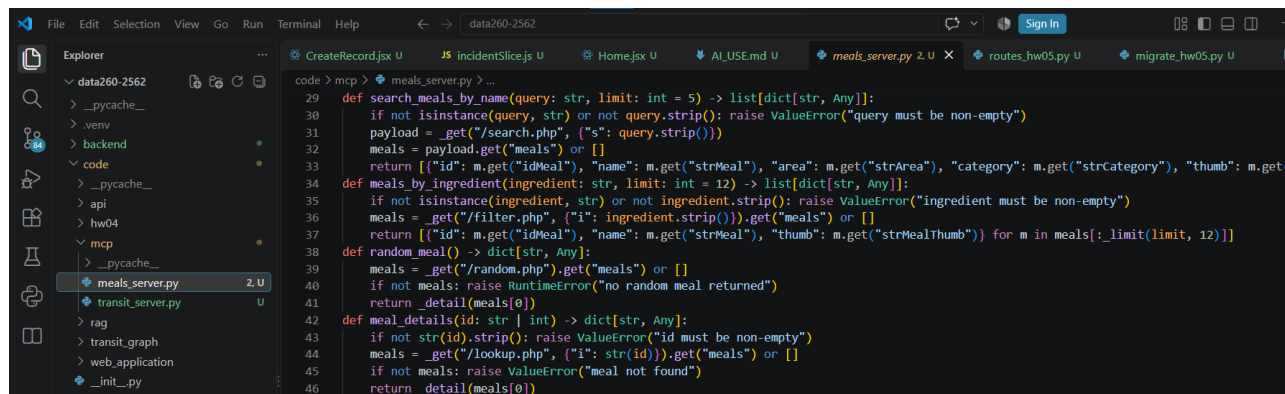
Part 2B - Rejected municipal transit detail lookup



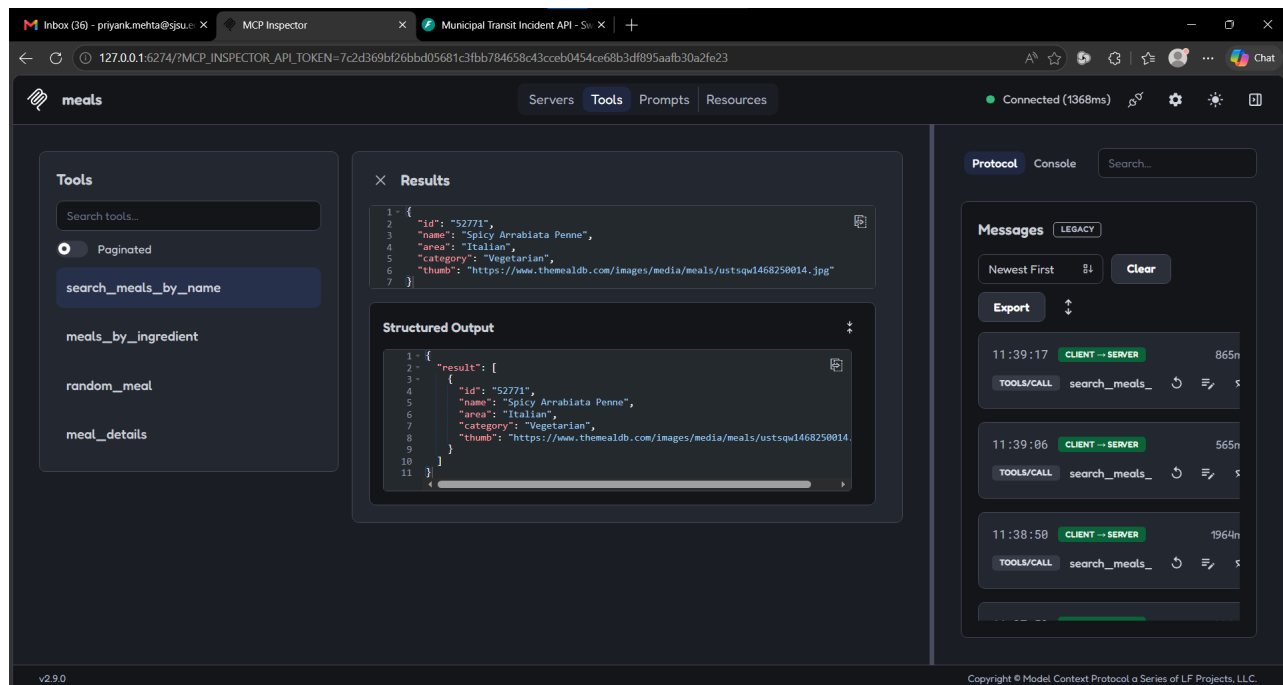
Part 2B - Successful municipal transit aggregate



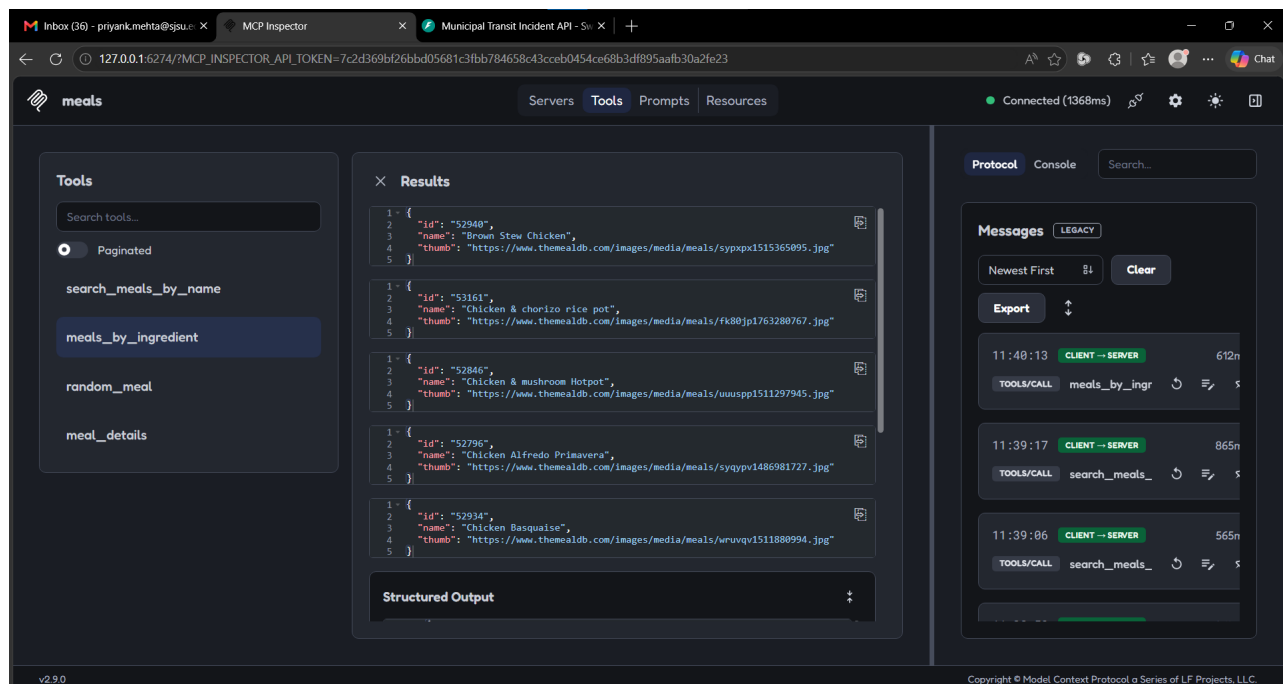
Part 2B - Rejected municipal transit aggregate



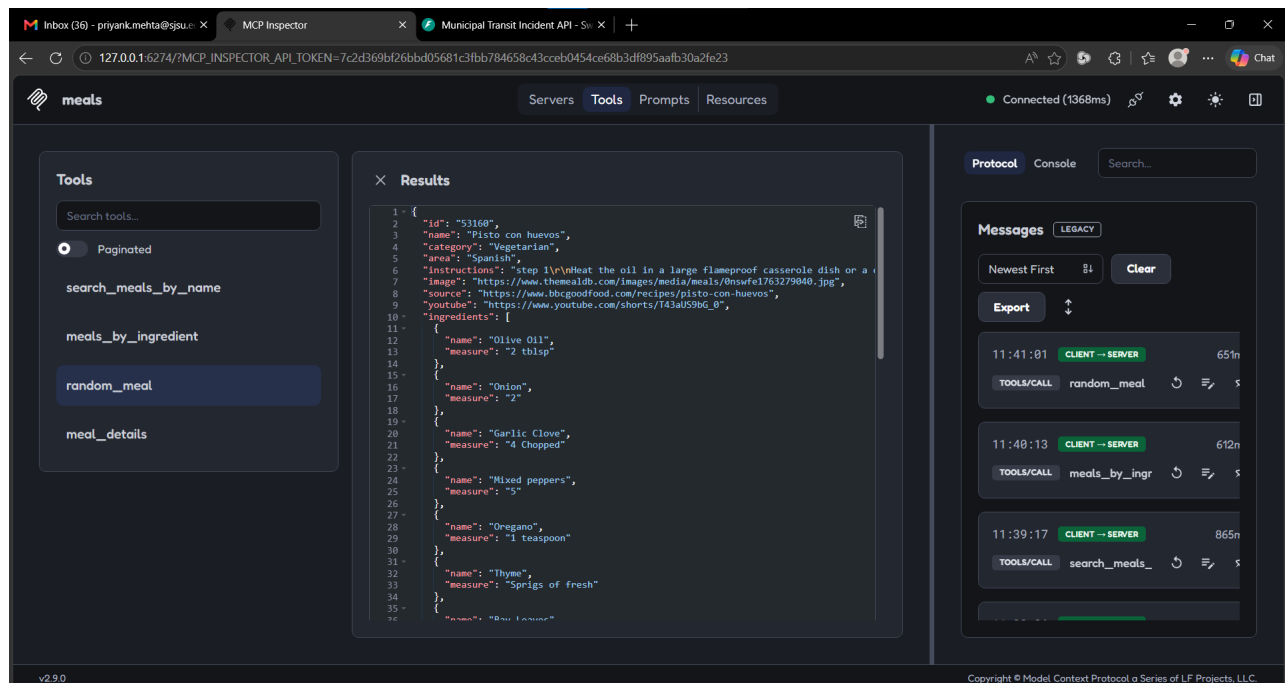
Part 2A - TheMealDB MCP server tool definitions and Inspector results



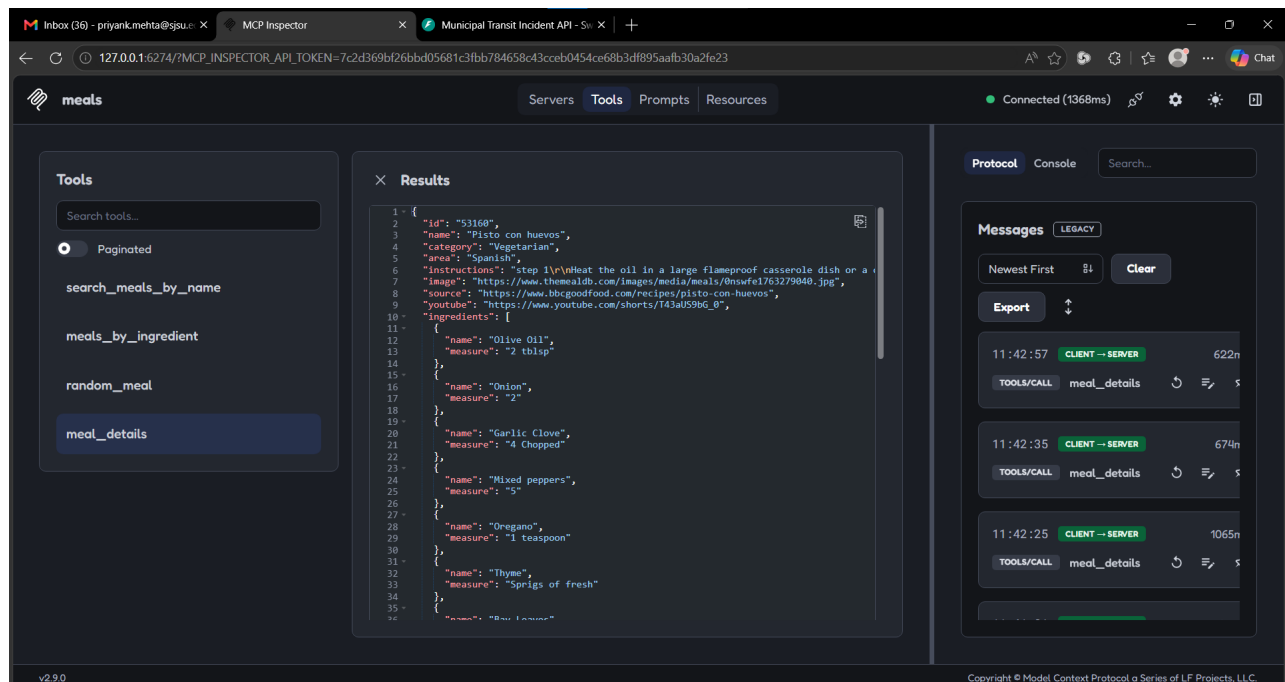
Part 2A - TheMealDB search by meal name



Part 2A - TheMealDB search by ingredient



Part 2A - TheMealDB random meal



Part 2A - TheMealDB meal details

4. Part 3 Tool Contracts and Retry Experiments

The domain tools validate input and return clean error envelopes. The retry policy uses timeouts, bounded exponential backoff, and deterministic failure injection with VERIFY_SEED. The experiment contains 150 raw calls: 50 at each of 0%, 20%, and 50% injected failure rates.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\MEHTA> cd "D:\Grad Prep\SJSU\GitHub\data260-2562"
>> Write-Output "Priyank Mehta - DATA 260 HW5 Part 4"
>> python code\hw05_tools.py
Priyank Mehta - DATA 260 HW5 Part 4
PASS search valid
PASS search invalid
PASS detail valid
PASS detail invalid
PASS aggregate valid
PASS aggregate invalid
PASS safety blocked
PASS max steps
8/8
PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

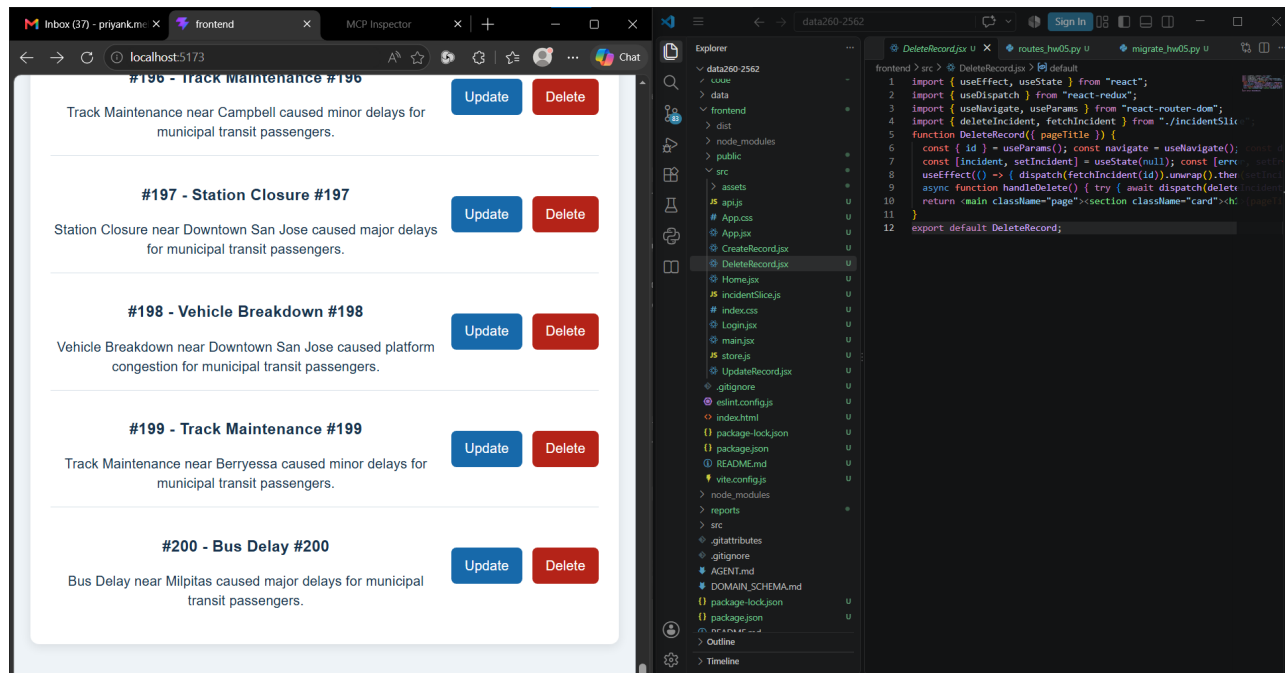
Part 3 - Deterministic retry policy and fault-injection experiment

```
code > hw05_tools.py > run_experiment
38 def retry_call(fn,rate,rng,attempts=3,backoff=0.001,timeout_s=2.0):
39     started=time.perf_counter()
40     for attempt in range(1,attempts+1):
41         try:
42             if rng.random()<rate: raise TimeoutError("injected timeout")
43             with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
44                 value=pool.submit(fn).result(timeout=timeout_s)
45             return True,attempt,(time.perf_counter()-started)*1000,value
46         except Exception as exc:
47             if attempt==attempts: return False,attempt,(time.perf_counter()-started)*1000,str(exc)
48             time.sleep(backoff*(2**(attempt-1)))
49     return False,attempts,(time.perf_counter()-started)*1000,"retry exhausted"
50 def run_experiment(out=ROOT/"reports/hw05/raw"):
51     out.mkdir(parents=True,exist_ok=True); records=[]
52     for rate in (0.0,0.2,0.5):
53         for call in range(50):
54             rng=random.Random(262562+int(rate*100))+call
55             ok,attempt,latency,value=retry_call(lambda: ("ok":True),rate,rng)
56             records.append(("rate":rate,"call":call+1,"success":ok,"attempts":attempt,"latency_ms":round(latency,4),"result":value))
57     (out/"fault_injection.json").write_text(json.dumps(records,indent=2),encoding="utf-8")
58     return records
```

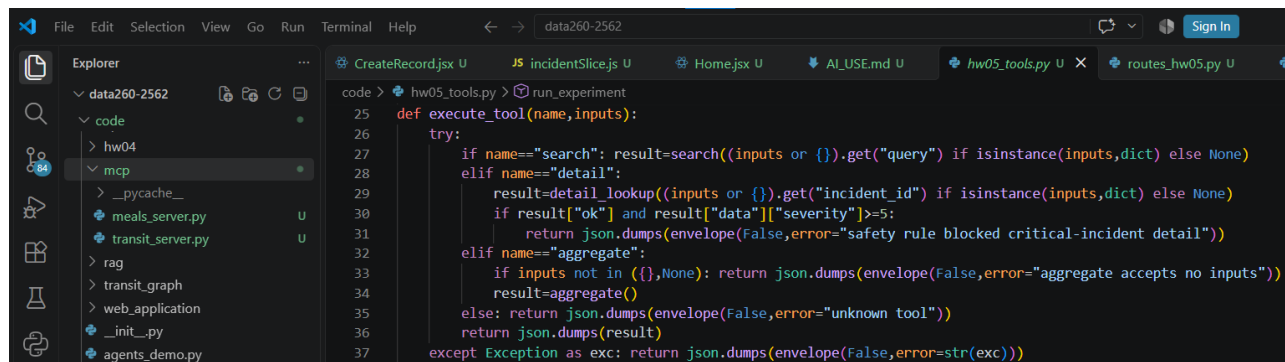
Part 3 - Retry implementation source code

5. Part 4 Safe Tool Entry Point and Offline Tests

`execute_tool` is the single JSON-returning entry point for all three domain tools. The offline test runner covers valid and invalid inputs, safety behavior, and `max_steps` behavior without requiring an LLM or database.



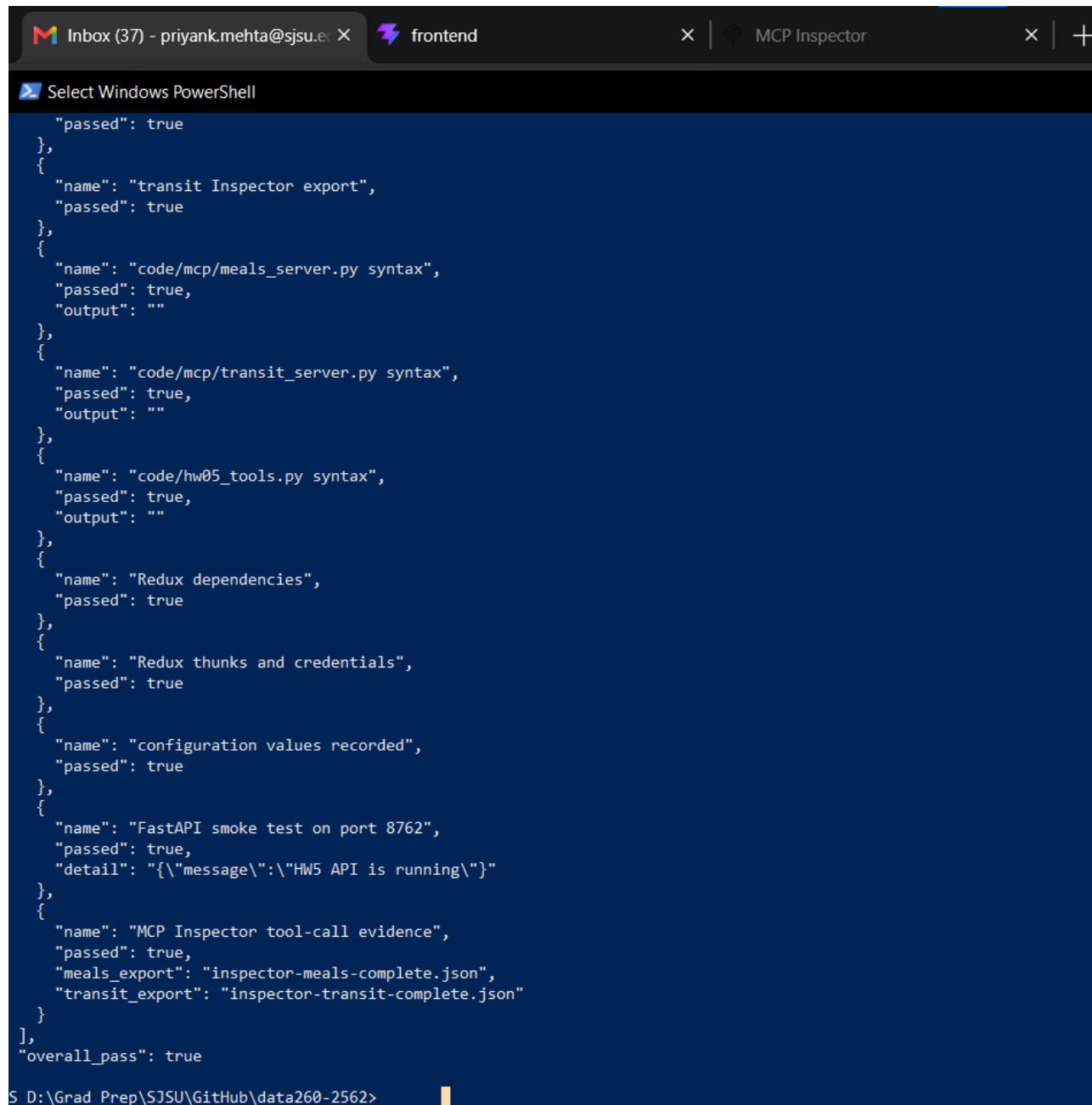
Part 4 - Safe `execute_tool` entry point and offline test runner



Part 4 - Offline `execute_tool` test runner

6. Part 5 Agent Loop Safety and Reflection

The agent uses exactly the three domain tools, applies a domain-specific safety rule, records turn and stop information, and limits execution with `max_steps`. Four local-model scenarios and the JSONL run log are included in the repository evidence.



```
"passed": true
},
{
  "name": "transit Inspector export",
  "passed": true
},
{
  "name": "code/mcp/meals_server.py syntax",
  "passed": true,
  "output": ""
},
{
  "name": "code/mcp/transit_server.py syntax",
  "passed": true,
  "output": ""
},
{
  "name": "code/hw05_tools.py syntax",
  "passed": true,
  "output": ""
},
{
  "name": "Redux dependencies",
  "passed": true
},
{
  "name": "Redux thunks and credentials",
  "passed": true
},
{
  "name": "configuration values recorded",
  "passed": true
},
{
  "name": "FastAPI smoke test on port 8762",
  "passed": true,
  "detail": "{\"message\": \"HW5 API is running\"}"
},
{
  "name": "MCP Inspector tool-call evidence",
  "passed": true,
  "meals_export": "inspector-meals-complete.json",
  "transit_export": "inspector-transit-complete.json"
}
],
"overall_pass": true
```

S D:\Grad Prep\SJSU\GitHub\data260-2562>

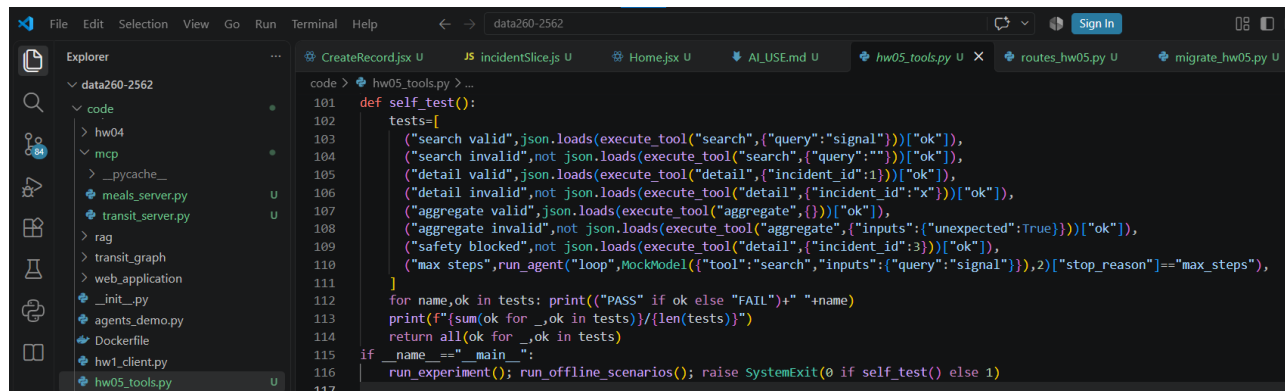
Part 5 - Safety rule, bounded agent loop, and JSONL logging

```
code > hw05_tools.py > run_offline_scenarios
74 def run_agent(user_input,model=None,max_steps=4,log_path=ROOT/"reports/hw05/raw/agent_runs.jsonl"):
75     model=model or MockModel()
76     steps=[]; stop="max_steps"
77     for turn in range(1,max_steps+1):
78         action=model.next_action(steps)
79         if action is None: stop="completed"; break
80         result=json.loads(execute_tool(action.get("tool"),action.get("inputs",{})))
81         steps.append({"turn":turn,"tool":action.get("tool"),"inputs":action.get("inputs",{}),"result":result})
82         if result.get("ok") and action.get("tool")=="aggregate": stop="completed"; break
83     record={"user_input":user_input,"steps":steps,"stop_reason":stop,"tool_call_count":len(steps)}
84     log_path.parent.mkdir(parents=True,exist_ok=True)
85     with log_path.open("a",encoding="utf-8") as f: f.write(json.dumps(record)+"\n")
86     return record
87 def run_offline_scenarios(out=ROOT/"reports/hw05/raw"):
88     out.mkdir(parents=True,exist_ok=True)
89     models=[MockModel({"tool":"search","inputs":{"query":"signal"}},MockModel({"tool":"detail","inputs":{"incident_id":3}}),MockModel({"
90     results=[run_agent(f"scenario-{i+1}",m,2,out/"agent_runs.jsonl") for i,m in enumerate(models)]
91     (out/"offline_agent_scenarios.json").write_text(json.dumps(results,indent=2),encoding="utf-8")
92     return results
93 def run_local_scenarios(out=ROOT/"reports/hw05/raw",model_name="qwen3:1.7b"):
94     out.mkdir(parents=True,exist_ok=True); results=[]
95     model=OllamaModel(model_name)
96     for i,prompt in enumerate(["Find signal incidents","Summarize transit totals","Look up incident 1","Find platform crowding"],1):
97         try: results.append(run_agent(prompt,model,4,out/"agent_runs.jsonl"))
98         except Exception as exc: results.append({"scenario":i,"user_input":prompt,"error":str(exc),"stop_reason":"unavailable"})
99     (out/"ollama_scenarios.json").write_text(json.dumps(results,indent=2),encoding="utf-8")
100     return results
```

Part 5 - Agent loop source code

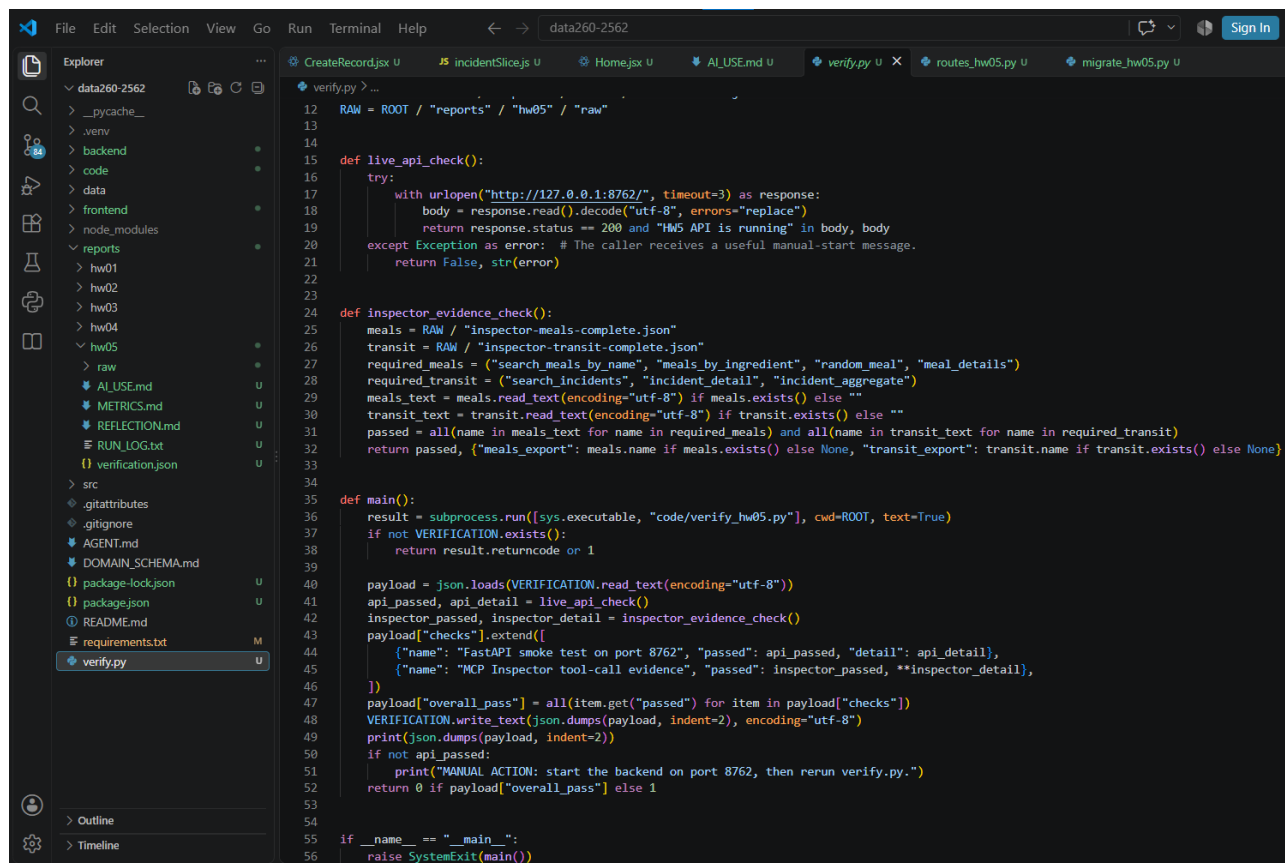
7. Verification and Reproducibility

The verification entry point runs the offline checks, confirms the raw-record counts and Inspector exports, checks Redux markers and syntax, and performs a live FastAPI smoke test when the backend is running on port 8762.



```
code > hw05_tools.py > ...
101 def self_test():
102     tests=[
103         ("search valid", json.loads(execute_tool("search", {"query": "signal"}))["ok"]),
104         ("search invalid", not json.loads(execute_tool("search", {"query": ""}))["ok"]),
105         ("detail valid", json.loads(execute_tool("detail", {"incident_id": 1}))["ok"]),
106         ("detail invalid", not json.loads(execute_tool("detail", {"incident_id": "x"}))["ok"]),
107         ("aggregate valid", json.loads(execute_tool("aggregate", {}))["ok"]),
108         ("aggregate invalid", not json.loads(execute_tool("aggregate", {"inputs": {"unexpected": True}}))["ok"]),
109         ("safety blocked", not json.loads(execute_tool("detail", {"incident_id": 3}))["ok"]),
110         ("max steps", run_agent("loop", MockModel({"tool": "search", "inputs": {"query": "signal"}}, 2))["stop_reason"]=="max_steps",
111     ]
112     for name, ok in tests: print(("PASS" if ok else "FAIL") + " " + name)
113     print(f"sum(ok for _, ok in tests)/len(tests)")
114     return all(ok for _, ok in tests)
115 if __name__ == "__main__":
116     run_experiment(); run_offline_scenarios(); raise SystemExit(0 if self_test() else 1)
117
```

Verification - HW5 self-check confirms the implementation and evidence



```
File Edit Selection View Go Run Terminal Help < -> data260-2562 Sign In
Explorer
data260-2562
  _pycache_
  .venv
  backend
  code
  data
  frontend
  node_modules
  reports
  hw01
  hw02
  hw03
  hw04
  hw05
    raw
    AI_USE.md
    METRICS.md
    REFLECTION.md
    RUN_LOG.txt
    verification.json
  src
    .gitattributes
    .gitignore
    AGENT.md
    DOMAIN_SCHEMA.md
    package-lock.json
    package.json
    README.md
    requirements.txt
    verify.py
  Outline
  Timeline

code > verify.py > ...
12 RAW = ROOT / "reports" / "hw05" / "raw"
13
14
15 def live_api_check():
16     try:
17         with urlopen("http://127.0.0.1:8762/", timeout=3) as response:
18             body = response.read().decode("utf-8", errors="replace")
19             return response.status == 200 and "HW5 API is running" in body, body
20     except Exception as error: # The caller receives a useful manual-start message.
21         return False, str(error)
22
23
24 def inspector_evidence_check():
25     meals = RAW / "inspector-meals-complete.json"
26     transit = RAW / "inspector-transit-complete.json"
27     required_meals = ("search_meals_by_name", "meals_by_ingredient", "random_meal", "meal_details")
28     required_transit = ("search_incidents", "incident_detail", "incident_aggregate")
29     meals_text = meals.read_text(encoding="utf-8") if meals.exists() else ""
30     transit_text = transit.read_text(encoding="utf-8") if transit.exists() else ""
31     passed = all(name in meals_text for name in required_meals) and all(name in transit_text for name in required_transit)
32     return passed, {"meals_export": meals.name if meals.exists() else None, "transit_export": transit.name if transit.exists() else None}
33
34
35 def main():
36     result = subprocess.run([sys.executable, "code/verify_hw05.py"], cwd=ROOT, text=True)
37     if not VERIFICATION.exists():
38         return result.returncode or 1
39
40     payload = json.loads(VERIFICATION.read_text(encoding="utf-8"))
41     api_passed, api_detail = live_api_check()
42     inspector_passed, inspector_detail = inspector_evidence_check()
43     payload["checks"].extend([
44         {"name": "FastAPI smoke test on port 8762", "passed": api_passed, "detail": api_detail},
45         {"name": "MCP Inspector tool-call evidence", "passed": inspector_passed, "detail": inspector_detail},
46     ])
47     payload["overall_pass"] = all(item.get("passed") for item in payload["checks"])
48     VERIFICATION.write_text(json.dumps(payload, indent=2), encoding="utf-8")
49     print(json.dumps(payload, indent=2))
50     if not api_passed:
51         print("MANUAL ACTION: start the backend on port 8762, then rerun verify.py.")
52     return 0 if payload["overall_pass"] else 1
53
54
55 if __name__ == "__main__":
56     raise SystemExit(main())
```

Verification - verify.py source code

8. Metrics and Results

The recorded retry measurements are summarized below.

Injected failure rate	Calls	Success rate	Mean latency (ms)	p99 latency (ms)
0%	50	100.0%	0.3893	4.0646
20%	50	100.0%	0.6679	4.7125
50%	50	92.0%	1.5600	5.0689

The retry policy is suitable for interactive use because it bounds waiting time and returns a clean failure after the retry budget is exhausted. Batch processing could use a larger retry budget and longer backoff delays because throughput and completion are more important than immediate response time.

9. AI Use Disclosure

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot environment setup and installation issues, including MCP Inspector setup, MCP version compatibility, PowerShell commands, Redux package installation, and frontend build errors.

I independently implemented and tested the application, API and MCP tools.

2. What AI-produced output was wrong or unsuitable?

An initially suggested MCP command was unsuitable for the installed environment. MCP Inspector launched the server with `uv`, but the project used a different Python environment and the server failed with `ModuleNotFoundError: No module named 'code.hw05_tools'; 'code' is not a package`.

An additional MCP test initially allowed invalid aggregate input and returned `"ok": true` instead of rejecting the request.

3. How did I detect the problem or verify the result?

I detected the MCP problem from the Inspector failure screen and the PowerShell traceback. The traceback showed that the server could not resolve the project module.

I detected the aggregate-validation problem by sending intentionally invalid JSON through MCP Inspector. The response incorrectly returned a successful envelope. I also verified frontend installation and build commands by checking the npm output, Python tracebacks, Inspector responses, and test-runner results.

4. What did I change, and why does it work now?

I corrected the MCP server import path so it can locate the project modules when launched directly by Inspector. I added compatibility for the installed MCP interface and moved diagnostic logging to `stderr` so `STDIO` responses remain valid.

I also changed the aggregate tool to accept an optional input object and explicitly reject unexpected input with `{ "ok": false, "data": null, "error": "aggregate accepts no inputs" }`.

For the frontend, I installed and configured Redux Toolkit and react-redux, preserved Axios credentials, and verified the Redux Home, Create, Update, and Delete features through the browser. I verified the final implementation with offline tests, MCP exports, API tests, and the required screenshots.

10. Conclusion

HW5 extends the existing municipal transit application across database relationships, validated CRUD APIs, Redux Toolkit state management, MCP tool servers, deterministic retry experiments, safe tool execution, and a bounded local-model agent. The captured evidence and machine-readable outputs are stored in reports/hw05.